

항공기 날개 공력탄성학 및 플러터 진동 해석

Aeroelasticity and Flutter Analysis of Aircraft Wings

과목번호 120520-1

학번 C417025

이름 김세경

제출일 June 22, 2026

1 시스템 모델 및 상태공간 정의

1.1 모델 간소화 및 자유도 축소

해석의 용이성과 자원의 한계를 고려하여 다음 가정 하에 진행하였다. 날개는 형상이 변하지 않는 대칭, 균일 구조이다; 이때 탄성에 의한 효과는 길이 방향만 고려하며, 날개의 길이는 무한하여 wing vortex와 같은 3차원 효과는 무시한다; 길이 방향 탄성효과는 익형에 달린 두 스프링과 두 댐퍼로 근사한다.(에어포일 형상은 탄성의 영향을 받지 않는 강체로 가정한다.) 실험은 풍동 내부를 가정하며 실제 날개와의 유사성은 Dynamic Similarity로 보장된다. 순항고도와 같이 비교적 steady한 상황의 미소 섭동(perturbation)에서 시스템은 선형시간불변(LTI)으로 간주 될 수 있다. [4], 마지막으로, 난류(gust)는 구조적 스트레스가 지배적인 수직 방향만 고려한다.

1.2 모델

날개의 거동학을 알기 위해 풍동 안에서 실험이 이루어진다고 하면, 날개의 익형(airfoil)이 일정하기 때문에 Chord length c_{chord} 는 상수다. 그리고 무한히 긴 날개에서, 동체에서 b 만큼 떨어진 날개의 단면에 대한 등가모형을 아래와 같은 strut-supported wind tunnel model로 가정할 수 있다.[2]

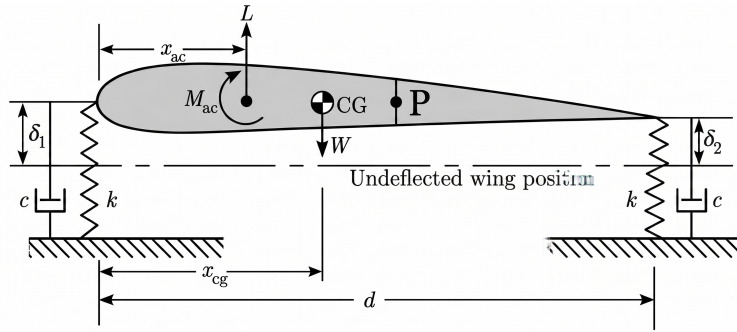


Figure 1: Cross section of strut-supported wind-tunnel model.
The wing is represented as a rigid airfoil on two spring-damper support

동일한 강성을 가진 스프링 k 가 d 만큼 떨어진 상태로 두개 위치하며, 각각의 스프링 옆에는 동일한 감쇠계수 c 를 가진 감쇠기가 위치한다. 익형의 중간지점 P 에 대해서, 아래와 같이 운동을 기술하기 위한 두개의 변수를 정의한다.

h — the plunge, 기준점 undeformed wing position에 대한 수직거리 (positive downward)

θ — the pitch, 익형의 회전 (positive nose-up).

임의의 chordwise location x 에 대한 vertical displacement z 는 아래와 같이 정의할 수 있다.

$$z = h + x\theta \quad (1)$$

해당 기준점 P 에 대하여, 익형의 시작점을 기준으로 위 그림과 같이 공력중심, 무게중심, 중점 순으로 위치하며, 기준점 P 에 대하여 P 와 무게중심까지의 거리를 x_{cg}^p 로, 공력중심까지의 거리를 x_{ac}^p 로 정의한다. 또한 무게중심과 공력중심 사이의 거리를 $d_{cg}^{ac} = x_{cg}^p - x_{ac}^p$ 로 정의하며, 이는 뒤의 플러터 해석에서 핵심 설계변수로 쓰인다.

1.3 운동방정식 유도

약 3 4주차에 걸쳐서 아래의 운동방정식을 유도하였다. 질량 $\mathbf{M}$, 구조 감쇠 $\mathbf{C}$, 구조 강성 $\mathbf{K}_s$ 는 라그랑지안에서, 공력 감쇠 $\mathbf{C}_a$, 공력 강성 $\mathbf{K}_a$, 돌풍 입력 벡터 $\mathbf{F}_g$ 는 준-정상상태¹ 공력 모델로부터 얻어진다. 각 행렬의 자세한 유도과정은 부록 appendix A에 수록하였다.

$$\mathbf{M} \ddot{\mathbf{q}} + (\mathbf{C} + \mathbf{C}_a) \dot{\mathbf{q}} + (\mathbf{K}_s + \mathbf{K}_a) \mathbf{q} = \mathbf{F}_g w_g(t). \quad (2)$$

여기서 $\mathbf{q} = [h, \theta]^T$ 는 plunge와 pitch를 모은 일반화 좌표이고, $w_g(t)$ 는 수직 돌풍 성분이다.

1.4 상태공간 정의

(2)은 $\mathbf{q}$ 에 대한 2차 미분방정식이므로, 상태벡터(시스템벡터)를 $\mathbf{x} = [\mathbf{q}, \dot{\mathbf{q}}]^T \in \mathbb{R}^4$ 로 정의하면, 위의 운동방정식은 상태공간으로 나타내 줄 수 있다. $\mathbf{K}_{eff} = \mathbf{K}_s + \mathbf{K}_a$, $\mathbf{D}_{tot} = \mathbf{C} + \mathbf{C}_a$ 로 두고 $\ddot{\mathbf{q}}$ 에 대해 정리한 뒤, 출력을 $\mathbf{y} = \mathbf{q} = [h, \theta]^T$ 라 하면,

$$\dot{\mathbf{x}} = \mathbf{A} \mathbf{x} + \mathbf{B} w_g, \quad \mathbf{y} = \mathbf{C}_{out} \mathbf{x} \quad (3)$$

¹quasi-steady

를 얻는다. 행렬 **A**, **B**, **C_{out}**의 구체적 형태와 변환 과정은 부록 appendix A.7에 정리하였다. 상태공간을 얇으므로 임의의 돌풍 입력 w_g 에 대한 익형의 변위·회전 응답을 예측할 수 있고, 시스템 행렬 **A**의 고유값·고유벡터로부터 안정성, 공진주파수, 모드형태를 분석할 수 있다.

1.4.1 모델 계수

여러 소스들[1, 4, 2]과 FAA에 공시되어져 있는 자료들을 통해서 NACA2412의 데이터를 혼합하면, 아래와 같은 풍동에서 실험한 데이터를 얻을 수 있다. 해당 계수를 베이스로 하여금 앞으로 분석을 진행할 것이다.

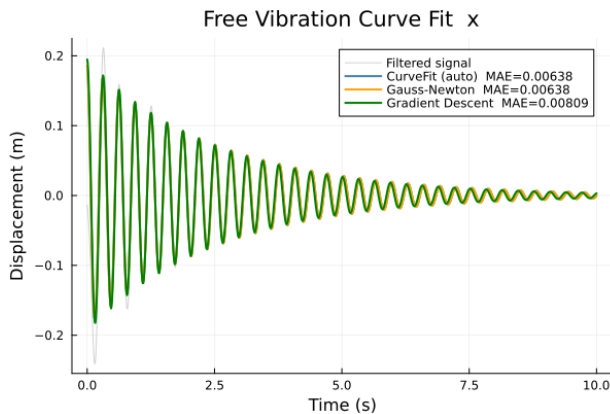
Table 1: Illustrative parameters (chosen to give two well-separated peaks).

Symbol	Value	Symbol	Value
m	1.50 kg	ρ	1.23 kg m ⁻³
I_p	0.008 40 kg m ²	U	10.0 m s ⁻¹
S	0.0225 kg m	c_{chord}	0.300 m
k (each)	900 N m ⁻¹	b_{span}	0.500 m
c (each)	1.00 N s m ⁻¹	$C_{l\alpha}$	6.00 rad ⁻¹
d	0.220 m	e_a	0.0450 m

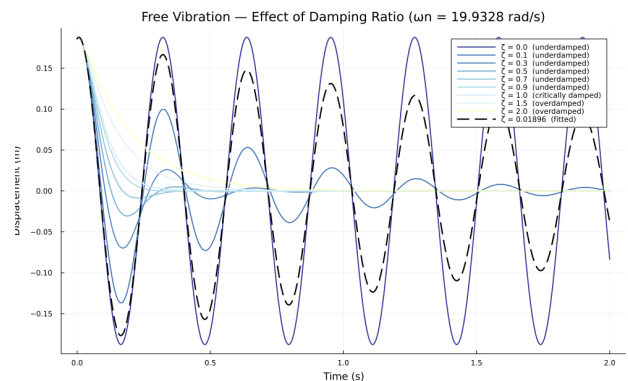
2 주차별 스토리텔링

2.1 curve-fitting을 이용한 등가모델(WEEK1)

1주차에서 우리는 날개를 1DOF mass-damp-spring 시스템²으로 설계해보았다. 센서 노이즈를 제거한 후, 여러 curve-fitting 알고리즘(Gradient Descent, Gauss-Newton etc)등을 통해 수치해석적 기법을 통해, 적당한 수준으로 모델링이 이루어졌을 때, 그에 상응하는 여러 진동학적 상수를 얻을 수 있음을 알 수 있었다. fig. 2 감쇠비가 1 이하이면 진폭이



(a) 여러 curve fitting algorithm을 이용한 등가모델 찾기
 $X = 0.18813, \zeta = 0.01896, w_n = 19.9328, \phi = 0.1525$



(b) damping ratio ζ 변화에 따른 진동 소산 현상의 변화.

$$\text{Figure 2: } x(t) = X e^{-\zeta w_n t} \cos(\sqrt{1 - \zeta^2} w_n t - \phi)$$

크고, 1 이상이면 진동소산까지의 시간이 오래걸림을 알 수 있다. 이 과정에서 감쇠비($\zeta=1$) 근처로 디자인을 해야함을 알 수 있다. 자재에 따라서 등가 구조 강성 k 가 바뀌면 자연주파수 $w_n = \sqrt{k/m}$ 이 증가하므로 진동을 더 빠르게 할 것이며, 이는 동일한 ζ 에 대해서 $w_d = \sqrt{1 - \zeta^2} w_n$ 이므로, 결과론적으로 transient state(초기 단계)에서 더 빠른 진동을 일으킬 것임을 알 수 있다.

2.2 가진과 질량 강성의 관계(WEEK2)

2주차에서는 1주차에서 모델링된 모델을 통해 임의의 가진에 대한 시스템의 반응, 그리고 질량 변화에 따른 시스템 거동의 변화에 대해 살펴 보았다. 아래와 같이 일반적인 소형항공기에서 많이 쓰이는 Rotary engine에 대한 진동에 의한 힘이 시스템에 전달 될 때 반응하는 거동을 살펴보면, 아래와 같이, 초반(transient)구간에서는 흥분현상으로 인해 큰 진폭을 보였으나, 갈수록 정상상태로 진입하여, 엔진의 진동수와 일치해가는 현상을 발견할 수 있었다. [3] 또한, 스프링 강성이 감소함에 따라, 1DOF의 고유주파수식, $w_n = \sqrt{k/m}$ 에 따라, 고유주기가 증가하는 모습을 볼 수 있었으며, 진동의 진폭 또한 증가하는 것을 확인 할 수 있었다. 진폭이 증가하는 현상은 질량이 증가함에 따라 어느정도

² $m\ddot{x} + c\dot{x} + kx = F$

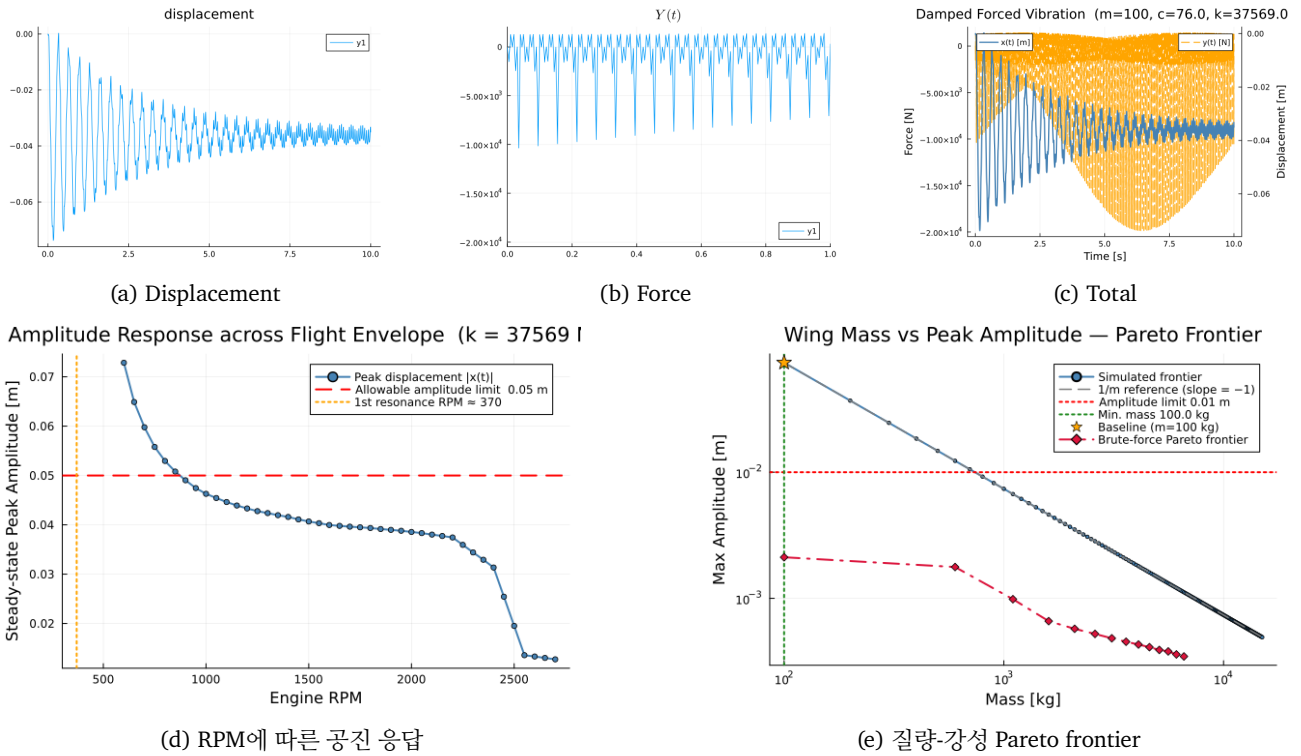


Figure 3: 2주차 결과

감소하는 것을 알 수 있었다. 이러한 관계(날개의 질량을 과도하게 줄이면 강성 또한 낮아지며 진동으로 인한 피로가 증가함)에서, 질량과 강성 사이에 최적의 경우를 찾아야 함을 알 수 있었고, 그것을 찾기 위해 Pareto frontier와 같은 최적화 기술이 쓰임을 알 수 있었다. 위에 보이는 사진들과 같이, w_n 을 고정시키고 탐구를 해보면, 질량을 증가시키면 시스템의 진폭이 감소하나, 이는 강성이 그만큼 증가해야 한다는 것을 의미한다는 것을 알 수 있다.

이러한 관계 속에서 항공기의 연료가 탱크가 아닌 날개에 먼저 저장되는 이유도 경량화의 목적을 이루면서 날개쪽에 질량을 늘려, 진동으로 피로를 줄이기 위함임을 유추 할 수 있었다. 마지막으로, 탐구 과정 도중, 위의 1DOF 모델은 아래 그림과 같이 상대적 저주파에서 공진현상을 보임을 알 수 있었다. 이로써 엔진의 과도한 idling이 연비는 아낄 수 있지만, 기체에 가하는 피로로 인해 득보다 실이 더 클 수도 있다는 것을 알게 되었다.

2.3 날개의 로우패스 필터 특성과 공명현상(WEEK3)

3주차부터는 위에서 서술한 2DOF 모델을 활용하여 탐구를 진행하였다. 난기류는 그 모델에 따라 von Kármán, Dryden turbulent model 등 다양한 모델이 존재한다. 본 탐구에서는 유리함수 형태로 다루기 편한 Dryden 수직 성분 PSD를 사용하였다.³ 이러한 모델에서 알 수 있는 특성으로는, 난기류는 저주파 부분에서 큰 진폭을 가지며, 고주파 부분은 거의 존재하지 않는다는 부분을 확인 할 수 있었다. 따라서 해당 근거에 따라서, 아래 그림과 같이 날개가 로우패스 필터의 특성을 보이고, 특정 주파수에서 진폭이 커지는 공진현상을 발견할 수 있었으므로, 날개를 설계할 때는 공명주파수를 너무 저주파는 아닌, 적당한 주파수 대역폭 사이에 잡아야 함을 알 수 있었다. 해당 탐구에서는 보다 더 확실한 날개의 로우패스 필터의 특성을 파악하기 위해, 여러개의 동일한 진폭을 가진 임의의 난기류 사인파를 만들어서, 그것에 대한 주파수 반응을 나타내보았다.

³Dryden vertical PSD: $\Phi_w(\omega) = \frac{\sigma_w^2 L_w}{\pi U} \frac{1+3(L_w \omega/U)^2}{[1+(L_w \omega/U)^2]^2}$, $\sigma_w = 1.5$, $L_w = 1.0$

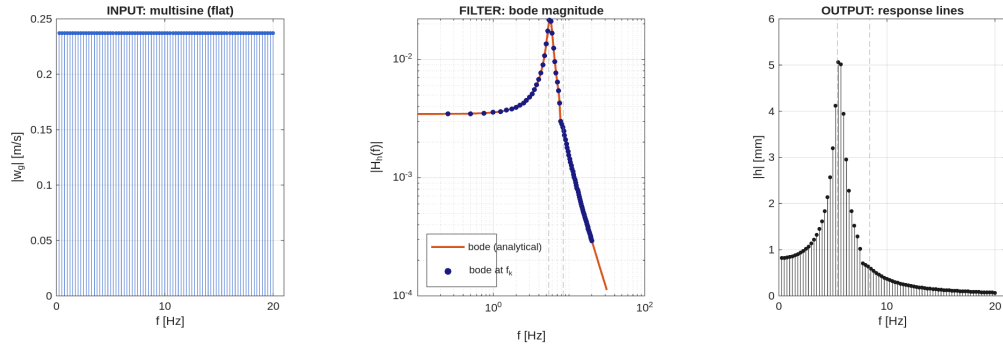
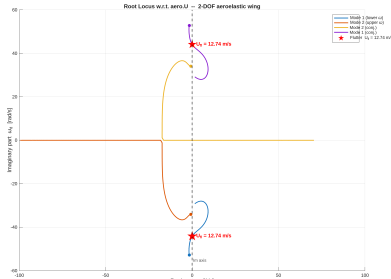


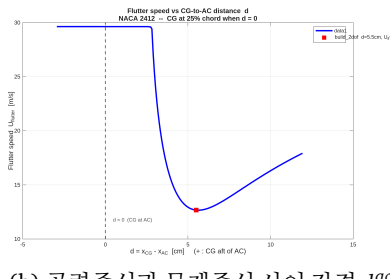
Figure 4: Frequency response of sine waves with rms=1.5

2.4 속도에 따른 고유값의 변화(WEEK5)

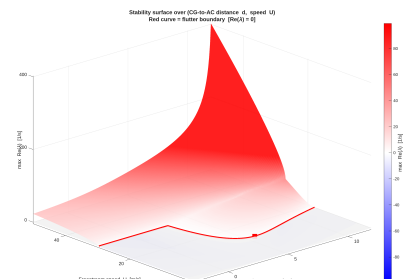
(2)에서, 공기력 행렬의 C_a, F_g 는 U 에 대한 함수임을 알 수 있다. 즉, 속도가 변함에 따라 system matrix A 가 변하므로, 평균 순항속도 U 에 따라서 시스템의 λ^4 가 달라진다. 해당 시스템의 두개의 모드는 고유값의 허수부분에, 감쇠는 실수에 연결되어져 있는것을 이용하기 root-locus plot를 사용하여 RHS를 넘어가는 고유값이 있나 살펴보았다.⁵ 보이는



(a) Root-Locus plot w.r.t. velocity U . ★ indicate the boundary of stable.



(b) 공력중심과 무게중심 사이 간격 d_{cg}^{ac} 에 따른 속도와 flutter boundary사이의 관계



(c) 공력중심과 무게중심 사이 간격 d_{cg}^{ac} 에 따른 속도와 고유값 사이의 관계

Figure 5: 5주차 결과. 빨간색 네모는 시스템이 table 1일 때 시스템이 위치하는 곳이다.

그림과 같이, 위의 값들은 날개의 진동분석을 위해 비교적 불안정한 위치에 설계된 날개 모델임을 알 수 있다. 이러한 불안정성은 공력중심과 무게중심 사이의 간격을 일정 수준 줄임으로써 fig. 5b 해소할 수 있음을 볼 수 있었다. 그리고 이렇게 d_{cg}^{ac} 를 바꾸었을 때에는, root locus의 모양또한 달라질 것이므로, 다시 해당 그래프를 그려서 flutter boundary 근처에서의 속도에 따른 고유값(λ)를 재조사하여 보다 안정적인 설계를 해야함 또한 알 수 있었다. 또한 위 경우에는 주파수 병합이 일어나는⁶, 동일한 속도에 대해서 두 고유값의 허수부분이 동일한 현상이 일어나지 않았지만, 다른 U 에 대해서는 해당 경우가 발생할 수 있으므로, 이를 피해서 설계를 해야함을 알 수 있었다.

2.5 선형 비선형 모델의 비교(WEEK4)

위에서는 선형적인 경우만을 다루었으나, week4에서는 용수철이 비선형적으로 거동할 경우에 대해 탐구하였다. 비선형 시스템에서는 위와 같이 행렬 형태의 state space로 표현하지 못하여 bode, root locus와 같은 거동의 해석이 선형 시스템만큼 쉽지 않다. 해서 해당 탐구에서는 간단하게 impulse⁷, multisine 함수를 수치해석기법(ode45)를 사용하여 선형과 비선형 거동의 차이를 아래와 같이 보였다.

⁴eigenvalue

⁵결계를 제외한, 두개의 고유한 고유값들 중 하나라도 root-locus plot 에서 RHS로 넘어가면 시스템은 발산한다

⁶두 모드가 동일한 고유진동수를 가지어, 동시에 공진현상이 발생해 큰 피해를 주는 현상

⁷FAA 14 CFR §25.341(a)

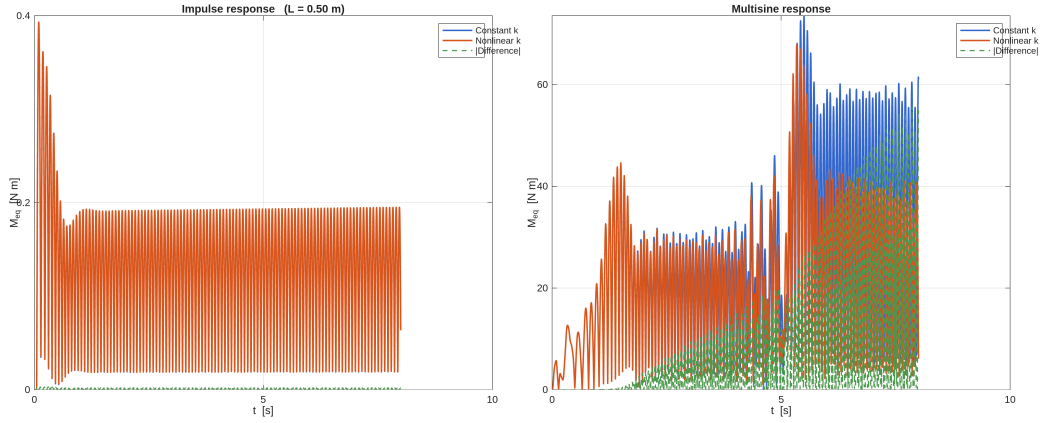


Figure 6: Impulse and multisine response of linear and nonlinear spring stiffness. velocity U is taken to flutter boundary speed to emphasize its different behavior; The cubic non-linear stiffness coefficient (k_{nl}) in $F_s = kx + k_{nl}x^3$ is set to 900. Others are same as above.

보이는 바와 같이, 초기단계에서는 두개가 별 차이가 없으나, 시간이 갈수록 그 차이가 점점 커짐을 알 수 있다. 또한 선형에 비해 비선형에서 모멘트가 커지므로, 비선형의 경우에서 피로 또는 항복응력 초과 확률이 더 높음을 알 수 있다. 하지만, 주기적으로 힘이 작용하는 sine 함수와 달리 impulse와 같은, 단기적으로 힘이 작용하는 경우에는, 선형, 비선형 스프링의 차이가 별로 없다는 점을 미뤄보아, 항공기의 속도가 빠르게 유지되어, 난기류 같은 순간적인 impulse로 작용하는 힘만 존재한다고 가정하면, 충분히 비선형 시스템은 선형 시스템으로 취급되어 해석을 할 수 있음을 알 수 있었다.

3 결론: 최종 설계안

5주에 걸친 해석을 종합하면, 안정적이면서 경량인 날개는 아래와 같은 최소 세가지 시선을 만족해야함을 알 수 있다. 1,2주차 질량&강성 trade-off에서 공진 진폭을 피로 한계 이내로 누르는 최소 강성 $k \approx 5500 \text{ N m}^{-1}$

3주차 주파수 응답 공진 대역폭을 난류 저주파 공진주파수 밖으로 밀어내는 w_d

5주차 플러터 한계를 최고 비행속도 $V_{\max}$ 이상으로 확보하기 위해 공력중심-무게중심 간격 d_{cg}^{ac}

4주차에서 보였듯, 지속적으로 몰아치는 드문 난기류의 경우를 제외한, 임펄스, 혹은 시간이 짧은 외부충격의 경우 비선형 효과가 충분히 작아, 선형효과만으로 설계 검증이 타당함을 위에서 보였다. 비선형 조건을 제외하고, 선형시스템에서 위 세가지 조건을 모두 만족하는 물리량들은 table 2에 정리하였다. 나오지 않은 물리량들은 table 1의 값들이다.

Table 2: 종합 해석으로부터 도출한 권장 설계 파라미터.

설계값	근거 (주차)	권장 방향 / 값
간격 d_{cg}^{ac}	플러터 한계 (5주)	$d_{cg}^{ac} \leq 2.52 \text{ cm}$ ($V_f = 29.6 \text{ m/s} > V_{\max}$)
공진주파수 ⁸ w_d	로우패스 대역 (3주)	$w_d \geq 13.6 \text{ Hz}$
강성 ⁹ k	질량-강성 trade-off (1·2주)	$k \gtrsim 5500 \text{ N m}^{-1}$
비선형 항	돌풍 응답 (4주)	순항조건에서 무시 가능

A Structural Model from Energy

A.1 Mass Matrix — Kinetic Energy

Let the section have mass m , centre of gravity a distance x_{cg}^p aft of P , and moment of inertia I_{cg} about the c.g. From (1), the downward velocity of the c.g. is $\dot{z}_{cg} = \dot{h} + x_{cg}^p \dot{\theta}$, so the kinetic energy is

$$T = \frac{1}{2}m(\dot{h} + x_{cg}^p \dot{\theta})^2 + \frac{1}{2}I_{cg}\dot{\theta}^2 = \frac{1}{2}m\dot{h}^2 + S\dot{h}\dot{\theta} + \frac{1}{2}I_p\dot{\theta}^2, \quad (4)$$

where the **static unbalance** and **inertia about P** are defined as

$$S \equiv m x_{cg}^p, \quad I_p \equiv I_{cg} + m (x_{cg}^p)^2. \quad (5)$$

⁹위에 서술한 Dryden PSD에 V_f 값 대입시 값 감소가 뚜렷해지는 지점을 $w_{d,crit}$ 로 지정하였다

Writing $T = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}}$ with $\mathbf{q} = [h, \theta]^T$ gives

$$\mathbf{M} = \begin{bmatrix} m & S \\ S & I_p \end{bmatrix} \quad (6)$$

A.2 Stiffness Matrix — Spring Potential Energy

The two springs sit at $\xi = -d/2$ (front) and $\xi = +d/2$ (rear). Their deflections are $z_1 = h - \frac{d}{2}\theta$ and $z_2 = h + \frac{d}{2}\theta$, so

$$V = \frac{1}{2}kz_1^2 + \frac{1}{2}kz_2^2 = \frac{1}{2}k \left[\left(h - \frac{d}{2}\theta \right)^2 + \left(h + \frac{d}{2}\theta \right)^2 \right] = kh^2 + \frac{1}{4}kd^2\theta^2. \quad (7)$$

The cross terms $\mp d h \theta$ cancel because P is the midpoint. With $V = \frac{1}{2} \mathbf{q}^T \mathbf{K}_s \mathbf{q}$,

$$\mathbf{K}_s = \begin{bmatrix} 2k & 0 \\ 0 & \frac{1}{2}kd^2 \end{bmatrix} \quad (8)$$

A.3 Damping Matrix — Rayleigh Dissipation

The dampers act in parallel with the springs, so the Rayleigh dissipation function is identical in form to (7) with $k \rightarrow c$ and $z \rightarrow \dot{z}$: $\mathcal{D} = \frac{1}{2}c\dot{z}_1^2 + \frac{1}{2}c\dot{z}_2^2$, giving¹⁰

$$\mathbf{C} = \begin{bmatrix} 2c & 0 \\ 0 & \frac{1}{2}cd^2 \end{bmatrix} \quad (9)$$

A.4 Structural Equations of Motion

Lagrange's equations, $\frac{d}{dt}(\partial T / \partial \dot{q}_i) - \partial T / \partial q_i + \partial V / \partial q_i + \partial \mathcal{D} / \partial \dot{q}_i = Q_i$, yield

$$\mathbf{M} \ddot{\mathbf{q}} + \mathbf{C} \dot{\mathbf{q}} + \mathbf{K}_s \mathbf{q} = \mathbf{Q}_{\text{aero}}, \quad (10)$$

where $\mathbf{Q}_{\text{aero}}$ denotes the generalised aerodynamic forces derived next.

A.5 Aerodynamic Forces

The lift acts at the aerodynamic centre, a distance e_a forward of P (i.e., $\xi_{\text{ac}} = -e_a$). A vertical gust of velocity w_g adds an incremental angle of attack w_g/U to the geometric pitch θ , where U is the flight speed. Furthermore, when the wing plunges downward at velocity $\dot{h}$, it experiences an upward relative wind, creating an induced angle of attack of $\frac{\dot{h}}{U}$. Under the quasi-steady approximation, lift is proportional to the effective angle of attack through $C_{l\alpha}$:

$$L = \bar{q}S C_{l\alpha} \left(\theta + \frac{\dot{h}}{U} + \frac{w_g}{U} \right), \quad \bar{q}S = \frac{1}{2}\rho U^2 c b \quad (11)$$

For a NACA 2412 airfoil, $C_{l\alpha} = \text{Const}(\approx 2\pi)$, and $\bar{q}S$ is the dynamic pressure multiplied by the planform area.¹¹ The virtual work of the upward lift under a virtual displacement of P gives the generalised forces:

$$Q_h = -L, \quad Q_\theta = +L e_a, \quad (12)$$

The constant aerodynamic-centre moment M_{ac} only shifts the trim and is dropped from the dynamics. Splitting (11) into a part driven by the motion $(\theta, \dot{h}, \dot{\theta})$ and a part driven by the gust w_g ,

$$\begin{bmatrix} Q_h \\ Q_\theta \end{bmatrix} = \underbrace{\bar{q}S C_{l\alpha} \begin{bmatrix} 0 & -1 \\ 0 & e_a \end{bmatrix} \begin{bmatrix} h \\ \theta \end{bmatrix} + \frac{\bar{q}S C_{l\alpha}}{U} \begin{bmatrix} -1 & 0 \\ e_a & 0 \end{bmatrix} \begin{bmatrix} \dot{h} \\ \dot{\theta} \end{bmatrix}}_{\text{motion-dependent}} + \underbrace{\frac{\bar{q}S C_{l\alpha}}{U} \begin{bmatrix} -1 \\ e_a \end{bmatrix} w_g}_{\text{gust forcing}}. \quad (13)$$

Moving the motion-dependent part to the left of (10) defines the **aerodynamic stiffness** $\mathbf{K}_a$, **aerodynamic damping** $\mathbf{C}_a$, and **gust force vector** $\mathbf{F}_g$:

$$\mathbf{K}_a = \bar{q}S C_{l\alpha} \begin{bmatrix} 0 & 1 \\ 0 & e_a \end{bmatrix}, \quad \mathbf{C}_a = \frac{\bar{q}S C_{l\alpha}}{U} \begin{bmatrix} 1 & 0 \\ -e_a & 0 \end{bmatrix}, \quad \mathbf{F}_g = \frac{\bar{q}S C_{l\alpha}}{U} \begin{bmatrix} -1 \\ e_a \end{bmatrix}. \quad (14)$$

Note that $\mathbf{K}_a$ is *not* symmetric: aerodynamic loads are non-conservative follower forces. This asymmetry is harmless at low speed but is the very mechanism that produces flutter at high speed.

¹⁰For a viscous damper $F_v = c\dot{z}$, the Rayleigh dissipation function is $\mathcal{D} = \frac{1}{2}c\dot{z}^2$, which represents half the rate of energy dissipation (power) of the system.

¹¹The unsteady Theodorsen/Sears corrections are omitted. This slightly overestimates the peaks but is appropriate for the present reduced-order model.

A.6 Assembled System

Collecting the structural matrices (6), (8), (9) and the aerodynamic matrices derived above, and moving the motion-dependent aerodynamic terms to the left-hand side, reproduces the governing equation stated in the main text, eq. (2), with $\mathbf{K}_{\text{eff}} = \mathbf{K}_s + \mathbf{K}_a$ and $\mathbf{D}_{\text{tot}} = \mathbf{C} + \mathbf{C}_a$.

A.7 State-Space Conversion

With the state vector $\mathbf{x} = [\mathbf{q}, \dot{\mathbf{q}}]^\top$, solving eq. (2) for $\ddot{\mathbf{q}}$,

$$\ddot{\mathbf{q}} = -\mathbf{M}^{-1}\mathbf{K}_{\text{eff}}\mathbf{q} - \mathbf{M}^{-1}\mathbf{D}_{\text{tot}}\dot{\mathbf{q}} + \mathbf{M}^{-1}\mathbf{F}_g w_g, \quad (15)$$

and stacking this with the trivial identity $\dot{\mathbf{q}} = \dot{\mathbf{q}}$ yields the first-order form $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}w_g$, $\mathbf{y} = \mathbf{C}_{\text{out}}\mathbf{x}$ with

$$\mathbf{A} = \begin{bmatrix} \mathbf{0}_{2 \times 2} & \mathbf{I}_2 \\ -\mathbf{M}^{-1}\mathbf{K}_{\text{eff}} & -\mathbf{M}^{-1}\mathbf{D}_{\text{tot}} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \mathbf{0}_{2 \times 1} \\ \mathbf{M}^{-1}\mathbf{F}_g \end{bmatrix}, \quad \mathbf{C}_{\text{out}} = [\mathbf{I}_2 \quad \mathbf{0}_{2 \times 2}], \quad \mathbf{D}_{\text{out}} = \mathbf{0}_{2 \times 1}. \quad (16)$$

The output matrix $\mathbf{C}_{\text{out}}$ selects the displacements $\mathbf{y} = [h, \theta]^\top$; other outputs (velocities, accelerations, or the root bending moment) are obtained by modifying $\mathbf{C}_{\text{out}}$ and $\mathbf{D}_{\text{out}}$ accordingly.

B 소스코드

B.1 저장소 구조

전체 소스코드는 아래 GitHub 저장소에 공개되어 있다.

https://github.com/sekyoung/2026-1_vibration_project

주차별 분석 스크립트는 `srcs/` 아래 주차 폴더에, 공통 유틸리티는 `srcs/` 루트에 위치한다.

*본 부록에 수록된 MATLAB 및 Julia 소스코드는 프로젝트의 최종 권장 설계안(Table 2)을 바탕으로 시스템의 동적 거동 및 플러터 경계를 최종 검증(Validation)하기 위해 작성되었습니다. 따라서 앞선 주차별 Baseline 파라미터(Table 1)와 수치적 차이가 있거나 변수명이 일부 상이할 수 있으며, 이는 최종 검증 시뮬레이션을 위한 독립적인 하드코딩 결과물입니다.

```
srcs/
├── build_models.m           % 선형/비선형 상태공간 행렬 생성 및 .mat 저장
├── modal_analysis.m        % 고유값·감쇠비 분석 및 극점 지도 출력
├── week1.jl                % 1주차: curve-fitting 등가모델 (Julia)
├── week2.jl                % 2주차: 가진 응답 분석 (Julia)
├── week2/
│   ├── fourier_torque.jl    % 로터리 엔진 토크의 푸리에 분해
│   └── tangential_force.jl  % 접선력 산출
├── week3/
│   ├── bode_response.m      % Bode 선도
│   ├── impulse_response.m   % 임펄스 응답
│   ├── multisine_analysis.m % 멀티사인 주파수 응답 분석
│   ├── multisine_function_generator.m
│   └── multisine_toolbox_response.m
├── week4/
│   ├── impulse_all.m        % 선형·비선형 임펄스 비교
│   ├── impulse_function_generator.m
│   ├── moment_response.m    % 굽힘 모멘트 응답
│   ├── multisine_function_generator.m
│   ├── nonlinear_response.m % 비선형 적분 (ode45)
│   └── result_plot.m
├── week5/
│   ├── flutter_vs_d.m       % CG-AC 간격에 따른 플러터 경계
│   └── root_locus_U.m       % 속도에 따른 root-locus
```

B.2 공통 유틸리티 코드

B.2.1 build_models.m

모든 주차 스크립트가 공유하는 .mat 데이터 파일을 생성하는 진입점이다. 내부적으로 두 함수를 호출한다: `build_2dof()`는 선형 2DOF 모델(`wing_2dof.mat`)을, `build_nl()`은 각 스프링에 3차 비선형 항을 포함한 모델(`wing_nl.mat`)을 조립하여 저장한다. 공통 공력 행렬($\mathbf{K}_a$, $\mathbf{C}_a$, $\mathbf{F}_g$)은 `aero_terms()`에서 한 번만 계산한 뒤 두 모델에 재사용된다.

```

1 function build_models()
2 % build_models Builds both Week-4 model files in this directory:
3 %   wing_2dof.mat -- linear 2-DOF model (Week-3 parameters, identical output)
4 %   wing_nl.mat   -- per-spring model with cubic stiffness + q=0 linearisation
5 %
6 % Replaces the need for week3/build_ss.m and week4/build_ss_nl.m at runtime.
7 %{
8 CF: Difference between build_models.m and the original build_ss.m and build_ss_nl.m:
9 build_models.m has two functions, build_2dof() and build_nl(), that construct the same physical models as
10 build_ss.m and build_ss_nl.m, respectively. The parameters
11 Comparing the two files, build_models.m's build_nl() function covers all the same physics and exports
12 identical variables to wing_nl.mat. But there is one missing piece:
13
14 Missing from build_models.m: the diagnostic fprintf statements that build_ss_nl.m prints (lines 73-75):
15
16 The full Ks matrix display (disp(Ks))
17 The coupling term report: Coupling term K_h,theta = ... (0 when k1 = k2)
18 build_models.m only prints the natural frequencies line, while build_ss_nl.m also prints the structural
19 stiffness matrix and the off-diagonal coupling term.
20
21 Everything else -- parameters, matrix assembly, state-space construction, saved variables -- is identical. If
22 those diagnostic prints matter to you, they can be added to build_nl().
23
24 %}
25 here = fileparts(mfilename('fullpath'));
26 build_2dof(here);
27 build_nl(here);
28 end
29
30 % =====
31 function [qS, Ka, Ca, Fgust, aero] = aero_terms()
32 aero.rho = 1.225; % air density [kg/m^3]
33 aero.U = 29.6; % freestream speed [m/s]
34 aero.chord = 0.30; % airfoil chord [m]
35 aero.span = 0.50; % section span [m]
36 aero.Clalpha = 6.0; % lift-curve slope [1/rad]
37 aero.ea = 0.045; % aero centre fwd of reference [m]
38
39 qS = 0.5 * aero.rho * aero.U^2 * aero.chord * aero.span;
40 Ka = [0, qS*aero.Clalpha;
41      0, -qS*aero.Clalpha*aero.ea];
42 Ca = (qS*aero.Clalpha/aero.U) .* [1, 0;
43      -aero.ea, 0];
44 Fgust = (qS*aero.Clalpha/aero.U) .* [-1; aero.ea];
45 end
46
47 % -----
48 function build_2dof(here)
49 % Week-3 linear model, lumped springs (k each side, symmetric).
50 m = 1.5; % section mass [kg]
51 Ip = 0.0084; % pitch inertia [kg*m^2]
52 S = 0.0225; % static unbalance [kg*m]
53 k = 5500; % each spring stiffness [N/m]
54 cd = 1.0; % each damper coefficient [N*s/m]
55 d = 0.22; % distance between springs [m]
56
57 [~, Ka, Ca, Fgust, aero] = aero_terms();
58
59 M_mat = [m, S;
60          S, Ip];
61 C_mat = [2*cd, 0;
62          0, cd*d^2/2];
63 Ks = [2*k, 0;
64       0, k*d^2/2];
65
66 Keff = Ks + Ka;
67 Ctotal = C_mat + Ca;
68
69 [A, B, C_out, D_out] = to_ss(M_mat, Keff, Ctotal, Fgust);
70 fn_struct = sqrt(eig(M_mat \ Ks)) / (2*pi);
71
72 U = aero.U; rho = aero.rho; chord = aero.chord;
73 span = aero.span; Clalpha = aero.Clalpha; ea = aero.ea;
74
75 save(fullfile(here, 'wing_2dof.mat'), ...
76      'A', 'B', 'C_out', 'D_out', ...
77      'M_mat', 'Keff', 'Ctotal', 'Fgust', ...
78      'U', 'rho', 'chord', 'span', 'Clalpha', 'ea', ...
79      'fn_struct');
80 fprintf('wing_2dof.mat fn = %.3f / %.3f Hz\n', fn_struct(1), fn_struct(2));
81 end
82
83 % -----
84 function build_nl(here)
85 % Per-spring model with cubic stiffness (cubic lives in the solver only).
86 m = 1.5; % section mass [kg]
87 Ip = 0.0084; % pitch inertia [kg*m^2]
88 S = 0.0225; % static unbalance [kg*m]
89 d = 0.22; % distance between springs [m]

```

```

88
89 k1 = 900.0; % spring 1 stiffness [N/m]
90 k2 = 900.0; % spring 2 stiffness [N/m] !(set != k1 for h-theta coupling)
91 knl1 = 1000; % spring 1 cubic coefficient [N/m^3] !(set > 0 for hardening)
92 knl2 = 1000; % spring 2 cubic coefficient [N/m^3]
93 c1 = 1; % damper 1 coefficient [N*s/m]
94 c2 = 1; % damper 2 coefficient [N*s/m]
95
96 r1 = +d/2; % spring/damper 1 moment arm [m]
97 r2 = -d/2; % spring/damper 2 moment arm [m]
98
99 [~, Ka, Ca, Fgust, aero] = aero_terms();
100
101 M_mat = [m, S;
102          S, Ip];
103 Ks = [k1 + k2, k1*r1 + k2*r2;
104       k1*r1 + k2*r2, k1*r1^2 + k2*r2^2];
105 C_struct = [c1 + c2, c1*r1 + c2*r2;
106             c1*r1 + c2*r2, c1*r1^2 + c2*r2^2];
107 C_mat = C_struct + Ca;
108
109 [A, B, C_out, D_out] = to_ss(M_mat, Ks + Ka, C_mat, Fgust);
110 fn_struct = sqrt(eig(M_mat \ Ks)) / (2*pi);
111 U = aero.U;
112
113 save(fullfile(here, 'wing_n1.mat'), ...
114       'A', 'B', 'C_out', 'D_out', 'fn_struct', ...
115       'M_mat', 'C_mat', 'Ka', 'Fgust', ...
116       'r1', 'r2', 'k1', 'k2', 'knl1', 'knl2', ...
117       'U');
118 fprintf('wing_n1.mat fn = %.3f / %.3f Hz\n', fn_struct(1), fn_struct(2));
119 end
120
121 % -----
122
123 function [A, B, C_out, D_out] = to_ss(M, K, C, Fgust)
124 Minv = inv(M);
125 A = [zeros(2), eye(2);
126      -Minv*K, -Minv*C];
127 B = [zeros(2,1);
128      Minv*Fgust];
129 C_out = [eye(2), zeros(2)];
130 D_out = zeros(2,1);
131 end

```

Listing 1: build_models.m — 상태공간 모델 생성

B.2.2 modal_analysis.m

wing_2dof.mat을 불러와 구조 고유값(A_{struct})과 공력탄성 고유값(A)을 각각 계산한다. 내부 함수 classify_eigs()가 켈레 복소 극점(부족감쇠)과 실수 극점(과감쇠·발산)을 자동으로 분류하여 고유진동수 f_n , 감쇠비 ζ 를 표로 출력하고 극점 지도를 그린다.

```

1 % modal_analysis.m -- Damping ratios and natural frequencies from wing_2dof.mat
2 %
3 % Two sets of results are computed:
4 % 1. Structural modes : eigenvalues of M_mat \ Ks (no aerodynamics)
5 % 2. Aeroelastic modes : eigenvalues of A (aero included)
6 %
7 % Eigenvalue types:
8 % Complex pair -> wn = |lambda|, zeta = -Re(lambda)/wn, fn = wn/(2*pi)
9 % Real negative -> overdamped / stable pole, reported as |lambda| [rad/s]
10 % Real positive -> divergence / unstable pole, reported as |lambda| [rad/s]
11
12 here = fileparts(mfilename('fullpath'));
13 load(fullfile(here, 'wing_2dof.mat'), ...
14       'A', 'M_mat', 'Keff', 'Ctotal', ...
15       'U', 'rho', 'chord', 'span', 'Clalpha', 'ea');
16
17 %% -- Reconstruct structural matrices -----
18 qS = 0.5 * rho * U^2 * chord * span;
19 Ka = [0, qS*Clalpha;
20       0, -qS*Clalpha*ea];
21 Ca = (qS*Clalpha/U) .* [1, 0;
22                        -ea, 0];
23
24 Ks = Keff - Ka;
25 C_struct = Ctotal - Ca;
26 Minv = inv(M_mat);
27
28 %% -- Build structural A-matrix and get all eigenvalues -----
29 A_struct = [zeros(2), eye(2);
30             -Minv*Ks, -Minv*C_struct];
31
32 lam_s = eig(A_struct);
33 lam_ae = eig(A);
34
35 %% -- Helper: classify and extract modal parameters -----

```

```

36 [s_wn, s_zeta, s_fn, s_type, lam_s_plot] = classify_eigs(lam_s);
37 [ae_wn, ae_zeta, ae_fn, ae_type, lam_ae_plot] = classify_eigs(lam_ae);
38
39 %% -- Print structural results -----
40 fprintf('\n=== Structural Modes (no aerodynamics) ===\n');
41 fprintf(' %-6s %-14s %-14s %-12s %s\n', ...
42         'Mode', 'fn [Hz]', 'wn [rad/s]', 'zeta [-]', 'Type');
43 for i = 1:numel(s_wn)
44     if isnan(s_zeta(i))
45         fprintf(' %-6d %14.4f %14.4f %12s %s\n', ...
46                 i, s_fn(i), s_wn(i), 'N/A', s_type{i});
47     else
48         fprintf(' %-6d %14.4f %14.4f %12.6f %s\n', ...
49                 i, s_fn(i), s_wn(i), s_zeta(i), s_type{i});
50     end
51 end
52
53 %% -- Print aeroelastic results -----
54 fprintf('\n=== Aeroelastic Modes (U = %.1f m/s) ===\n', U);
55 fprintf(' %-6s %-14s %-14s %-12s %s\n', ...
56         'Mode', 'fn [Hz]', 'wn [rad/s]', 'zeta [-]', 'Type');
57 for i = 1:numel(ae_wn)
58     if isnan(ae_zeta(i))
59         fprintf(' %-6d %14.4f %14.4f %12s %s\n', ...
60                 i, ae_fn(i), ae_wn(i), 'N/A', ae_type{i});
61     else
62         fprintf(' %-6d %14.4f %14.4f %12.6f %s\n', ...
63                 i, ae_fn(i), ae_wn(i), ae_zeta(i), ae_type{i});
64     end
65 end
66 fprintf('\n');
67
68 %% -- Store results -----
69 results.structural.fn_Hz = s_fn;
70 results.structural.wn_rads = s_wn;
71 results.structural.zeta = s_zeta;
72 results.structural.type = s_type;
73
74 results.aeroelastic.fn_Hz = ae_fn;
75 results.aeroelastic.wn_rads = ae_wn;
76 results.aeroelastic.zeta = ae_zeta;
77 results.aeroelastic.type = ae_type;
78
79 %% -- Pole map -----
80 figure('Name','Pole Map','Position',[100 100 620 460]);
81 hold on; grid on;
82
83 % structural: plot both upper and mirrored lower half
84 s_conj = [lam_s_plot; conj(lam_s_plot(imag(lam_s_plot) ~= 0))];
85 ae_conj = [lam_ae_plot; conj(lam_ae_plot(imag(lam_ae_plot) ~= 0))];
86
87 scatter(real(s_conj), imag(s_conj), 70, 'r', '^', 'LineWidth', 1.5, ...
88         'DisplayName', 'Structural');
89 scatter(real(ae_conj), imag(ae_conj), 70, 'b', 'o', 'filled', ...
90         'DisplayName', 'Aeroelastic');
91
92 xline(0, 'k--', 'LineWidth', 0.8, 'HandleVisibility', 'off');
93 yline(0, 'k:', 'LineWidth', 0.5, 'HandleVisibility', 'off');
94 xlabel('Real [1/s]');
95 ylabel('Imag [rad/s]');
96 title(sprintf('Pole Map (U = %.1f m/s)', U));
97 legend('Location', 'best');
98
99 %% -- Local function -----
100 function [wn, zeta, fn, type, lam_keep] = classify_eigs(lam)
101 % Separates complex (underdamped) and real (overdamped/unstable) eigenvalues.
102 % Returns one entry per unique mode.
103 tol = max(abs(lam)) * 1e-8;
104
105 is_complex = abs(imag(lam)) > tol;
106 lam_c = lam(is_complex & imag(lam) > 0); % upper half-plane of conjugate pairs
107 lam_r = real(lam(~is_complex)); % real poles
108
109 % sort: underdamped by ascending frequency, real by ascending value
110 [~, idx] = sort(abs(lam_c));
111 lam_c = lam_c(idx);
112 lam_r = sort(lam_r);
113
114 wn_c = abs(lam_c);
115 zeta_c = -real(lam_c) ./ wn_c;
116 fn_c = wn_c / (2*pi);
117 type_c = cell(numel(lam_c), 1);
118 for k = 1:numel(lam_c)
119     if zeta_c(k) >= 0
120         type_c{k} = 'underdamped (stable)';
121     else
122         type_c{k} = 'underdamped (unstable/flutter)';
123     end
124 end
125
126 wn_r = abs(lam_r);
127 fn_r = wn_r / (2*pi);

```

```

128 zeta_r = nan(numel(lam_r), 1);
129 type_r = cell(numel(lam_r), 1);
130 for k = 1:numel(lam_r)
131     if lam_r(k) < 0
132         type_r{k} = 'real (overdamped/stable)';
133     else
134         type_r{k} = 'real (divergent/unstable)';
135     end
136 end
137
138 wn = [wn_c(:); wn_r(:)];
139 zeta = [zeta_c(:); zeta_r(:)];
140 fn = [fn_c(:); fn_r(:)];
141 type = [type_c; type_r];
142 lam_keep = [lam_c(:); lam_r(:)];
143 end

```

Listing 2: modal_analysis.m — 고유값 분석 및 극점 지도

B.3 주차별 Pluto 노트북 (Julia)(WEEK1,WEEK2)

Pluto.jl은 Julia 기반의 반응형 노트북 환경으로, 각 셀은 파일 내에서 # UUID 주석으로 구분된다. 아래 코드에서 해당 주석이 회색으로 표시되어 셀 경계 역할을 한다. 패키지 의존성 잠금 파일(PLUTO*_TOML_CONTENTS)은 자동 생성 코드이므로 제외하였다. julia를 다운받아서 pluto를 통해 여는것이 제일 추천되는 방식이다.

B.3.1 week1.jl — 1주차: curve-fitting 등가모델

CSV.jl로 Ground Vibration Test 데이터를 불러온 뒤 Butterworth 대역통과 필터를 적용하고, Gradient Descent / Gauss-Newton / CurveFit 세 알고리즘으로 자유진동 해 $x(t) = X e^{-\zeta \omega_n t} \cos(\omega_d t - \phi)$ 의 파라미터를 피팅한다. 마지막 셀에서는 감쇠비 ζ 를 sweep하여 임계감쇠 근방 거동의 차이를 시각화한다.

```

week1.jl
1  ### A Pluto.jl notebook ###
2  # v0.20.25
3
4  using Markdown
5  using InteractiveUtils
6
7  # 44099108-4887-45f5-818c-8c1e258249f0
8  using CSV, DataFrames, Plots, FFTW, DSP, CurveFit, ForwardDiff, Statistics, LinearAlgebra, LaTeXStrings, PlutoUI
9
10 # f3d44c6a-0090-43a2-93e1-f2cdd16bb6cd
11 md"""
12 # WEEK 1
13 """
14
15 # 2c6c9ec3-2c74-4dd6-aea3-5481301d9185
16 df = CSV.read("./resources/week1/week1_wing_gvt_data.csv", DataFrame);
17
18 # b1041ed1-092f-4ecb-832a-95389f058ba1
19 begin
20     t = df[:, "Time (s)"]
21     x = df[:, "Displacement (m)"]
22
23     plot(t, x,
24          xlabel = "Time (s)",
25          ylabel = "Displacement (m)",
26          title = "Week 1 Wing GVT - Free Vibration",
27          legend = false,
28          lw = 1.5,
29          color = :steelblue
30     )
31 end
32
33 # d9433d2e-1be4-4846-917d-164cc5fb88b6
34 begin
35     N = length(t)
36     dt = t[2] - t[1]
37     fs = 1.0 / dt # sampling frequency (Hz)
38     # --- Step 1: FFT spectrum (before filter) ---
39     X = abs.(fft(x))[1:N÷2]
40     freqs = (0:N÷2-1) .* (fs / N)
41
42     dominant_freq = freqs[argmax(X)]
43     # --- Step 2: Butterworth band-pass filter ---
44     margin = 2.0
45     f_low = max(0.1, dominant_freq - margin)
46     f_high = dominant_freq + margin
47
48     nyquist = fs / 2.0
49     bpf = digitalfilter(Bandpass(f_low / nyquist, f_high / nyquist), Butterworth(4))
50     x_filtered = filtfilt(bpf, x)
51     # --- Step 3: Spectrogram ---
52     win_size = 128

```

```

54     overlap = win_size ÷ 2
55     sg      = spectrogram(x, win_size, overlap; fs=fs)
56             "Dominant frequency: $(round(dominant_freq, digits=3)) Hz ( $\omega_d = $(round(dominant_freq*2\pi, digits=3)) rad/s)$ "
57
58 end
59
60 # b4bd090d-e66a-4b00-8803-ae9e40a0267
61 begin
62     # --- Plot 1: Raw vs Filtered ---
63     p1_ = plot(t, x,
64               label = "Raw",
65               xlabel = "Time (s)", ylabel = "Displacement (m)",
66               title = "GVT Signal - Raw vs Filtered",
67               lw = 1.5, alpha = 0.8, color = :gray)
68     plot!(p1_, t, x_filtered,
69           label = "Butterworth BP ($(round(f_low, digits=1))-$$(round(f_high, digits=1)) Hz, order 4)",
70           lw = 2.0, color = :steelblue)
71
72     # --- FFT spectrum after filter ---
73     X_filtered = abs.(fft(x_filtered))[1:N÷2]
74
75     # --- Plot 2: FFT Spectrum before & after filter ---
76     xlim_max = min(fs/2, dominant_freq * 5)
77     p2_ = plot(freqs, X,
78               xlabel = "Frequency (Hz)", ylabel = "Amplitude",
79               title = "FFT Spectrum - Before vs After Filter",
80               legend = true, lw = 1.5, color = :darkorange,
81               xlims = (0, xlim_max),
82               label = "Before filter")
83     plot!(p2_, freqs, X_filtered,
84           lw = 1.5, color = :steelblue, alpha = 0.8,
85           label = "After filter")
86     vline!(p2_, [dominant_freq],
87            linestyle = :dash, color = :red, lw = 1.5,
88            label = "Dominant: $(round(dominant_freq, digits=2)) Hz")
89     vline!(p2_, [f_low, f_high],
90            linestyle = :dot, color = :gray, lw = 1.0,
91            label = "Band-pass range")
92
93     # --- Plot 3: Spectrogram ---
94     p3_ = heatmap(sg.time, sg.freq, pow2db.(sg.power),
95                  xlabel = "Time (s)", ylabel = "Frequency (Hz)",
96                  title = "Spectrogram",
97                  color = :inferno,
98                  ylims = (0, min(fs/2, dominant_freq * 5)),
99                  clim = (maximum(pow2db.(sg.power)) - 60, maximum(pow2db.(sg.power))))
100    hline!(p3_, [dominant_freq],
101           linestyle = :dash, color = :cyan, lw = 1.5,
102           label = "Dominant freq")
103
104    p_ = plot(p1_, p2_, p3_, layout = (3, 1), size = (1000, 1000))
105
106 end
107
108 # 98d8c9f2-36bf-4e8b-af3c-d4c7b729c4ea
109 md"""
110 If we neglect bending and torsional coupling, such caused by structural couple, sweep effects etc. then approximate wing as simple
111 ⇨ as simple beam, moving along with vertical direction. from elastic theory, its well known
112 where  $\delta = \frac{Wl^3}{3EI}$  . a typical airfoil, assume *NACA2412* in this case, has maximum 0.12 ratio of horizontal
113 ⇨ and vertical length. letting  $h=0.12b$  and  $k = \frac{3EI}{l^3}$ .
114
115 Therefore, its reducing to first order free vibration analysis.
116
117  $m\ddot{x} + c\dot{x} + kx = \sum_{i=1}^{\infty} H(i\omega) e^{-i\omega t} = 0$ 
118
119 the solution  $x(t)$ , is given as plot above.
120
121 # 9bcbce35-937b-494b-9a59-76c5d9a63cbb
122 md"""
123 by conventional usage of constant in vibration theory, we assume these constant
124
125  $\zeta = \frac{c}{c_c}$ 
126
127  $w_n = \sqrt{\frac{m}{k}}$ 
128
129  $c_c = 2mw_n$ 
130
131  $w_d = w_n \sqrt{1-\zeta^2}$ 
132
133 The following graph is showing the damping is underdamped, therefore we can express the displacement  $x(t)$  in form of
134
135  $x(t) = X e^{-\zeta w_n t} \cos(\sqrt{1-\zeta^2} w_n t)$ 
136
137 Using curve-fitting algorithms,
138
139 """

```

```

144
145
146 # 7443122e-1607-430c-99d0-b7d5a2984c7f
147 begin
148     t_arr=collect(t)
149     x_arr = x_filtered
150     f_dom = dominant_freq
151 # ----- Free-vibration model -----
152 # p = [X_amp, ζ, ωn, φ]
153 function fv_model(p, t_vec)
154     X_amp, zeta, wn, phi = p[1], p[2], p[3], p[4]
155     wd = sqrt(max(1.0 - zeta^2, 1e-10)) * wn
156     return @. X_amp * exp(-zeta * wn * t_vec) * cos(wd * t_vec - phi)
157 end
158
159 p0 = [maximum(abs.(x_arr)), 0.05, 2π * f_dom, 0.0]
160 loss(p) = sum((fv_model(p, t_arr) .- x_arr).^2)
161 mae(p) = mean(abs.(fv_model(p, t_arr) .- x_arr))
162 mse(p) = mean((fv_model(p, t_arr) .- x_arr).^2)
163
164 results = Dict{String, Vector{Float64}}{ }
165 errors = Dict{String, Float64}{ }
166 mse_errors = Dict{String, Float64}{ }
167
168 # ----- Algorithm 1: CurveFit.jl (NonlinearSolve auto-polyalgorithm) -----
169 try
170     prob = NonlinearCurveFitProblem(fv_model, p0, t_arr, x_arr)
171     sol = solve(prob)
172     results["CurveFit (auto)"] = sol.u
173     errors["CurveFit (auto)"] = mae(sol.u)
174     mse_errors["CurveFit (auto)"] = mse(sol.u)
175     println("CurveFit converged=$(isconverged(sol)) MAE=$(round(errors["CurveFit (auto)], digits=7))")
176 catch e
177     println("CurveFit failed: $e")
178 end
179
180 # ----- Algorithm 2: Gauss-Newton (ForwardDiff Jacobian + Tikhonov) -----
181 function gauss_newton(model, t_vec, y, p_init; maxiter=500, tol=1e-10)
182     p = copy(float.(p_init))
183     for _ in 1:maxiter
184         r = model(p, t_vec) .- y
185         J = ForwardDiff.jacobian(pp -> model(pp, t_vec), p)
186         dp = (J'J + 1e-8 * I) \ (J' * r)
187         p .-= dp
188         norm(dp) < tol && break
189     end
190     return p
191 end
192
193 try
194     p_gn = gauss_newton(fv_model, t_arr, x_arr, p0)
195     results["Gauss-Newton"] = p_gn
196     errors["Gauss-Newton"] = mae(p_gn)
197     mse_errors["Gauss-Newton"] = mse(p_gn)
198     println("Gauss-Newton MAE=$(round(errors["Gauss-Newton"], digits=7))")
199 catch e
200     println("Gauss-Newton failed: $e")
201 end
202
203 # ----- Algorithm 3: Gradient Descent with Armijo backtracking -----
204 function gradient_descent(loss_fn, p_init; maxiter=3000, tol=1e-8)
205     p = copy(float.(p_init))
206     for _ in 1:maxiter
207         g = ForwardDiff.gradient(loss_fn, p)
208         norm(g) < tol && break
209         lr = 1e-5
210         L0 = loss_fn(p)
211         for _ in 1:30
212             loss_fn(p .- lr .* g) < L0 - 1e-4 * lr * dot(g, g) && break
213             lr *= 0.5
214         end
215         p .-= lr .* g
216     end
217     return p
218 end
219
220 try
221     p_gd = gradient_descent(loss, p0)
222     results["Gradient Descent"] = p_gd
223     errors["Gradient Descent"] = mae(p_gd)
224     mse_errors["Gradient Descent"] = mse(p_gd)
225     println("Gradient Descent MAE=$(round(errors["Gradient Descent"], digits=7))")
226 catch e
227     println("Gradient Descent failed: $e")
228 end
229
230 # ----- Print fitted constants -----
231 println("\n=== Fitted Constants ===")
232 println("Parameter: [X_amp, ζ, ωn (rad/s), φ]")
233 for (name, p) in results
234     println("$name")

```

```

235     println(" X=$(round(p[1],digits=5))   ζ=$(round(p[2],digits=5))   ωn=$(round(p[3],digits=4)) rad/s
↪     fn=$(round(p[3]/(2π),digits=3)) Hz   φ=$(round(p[4],digits=4)) rad   wd=$(round(p[3]*√(1-p[2]^2),digits=4)) rad/s")
236     println(" MAE = $(round(errors[name], digits=7)) m")
237 end
238
239 # — Plot 1: Fitted curves vs filtered signal —————
240 algo_colors = ["CurveFit (auto)" => :steelblue,
241               "Gauss-Newton"    => :orange,
242               "Gradient Descent"=> :green]
243
244 p1 = plot(t_arr, x_arr,
245          label = "Filtered signal",
246          color = :gray, alpha = 0.35, lw = 1.0,
247          xlabel = "Time (s)", ylabel = "Displacement (m)",
248          title = "Free Vibration Curve Fit  x(t)=Xe^{-ζωt}cos(ωdt-φ)")
249
250 for (name, col) in algo_colors
251     haskey(results, name) || continue
252     plot!(p1, t_arr, fv_model(results[name], t_arr),
253          label = "$name MAE=$(round(errors[name],digits=5))",
254          lw = 2.0, color = col)
255 end
256
257 # — Plot 2 & 3: MAE and MSE bar charts side-by-side —————
258 names_sorted = sort(collect(keys(errors)))
259 mae_vals     = [errors[n] for n in names_sorted]
260 mse_vals     = [mse_errors[n] for n in names_sorted]
261 bar_cols     = [get(Dict{algo_colors}, n, :gray) for n in names_sorted]
262
263 p2 = bar(names_sorted, mae_vals,
264          xlabel = "Algorithm", ylabel = "MAE (m)",
265          title = "Mean Absolute Error",
266          color = bar_cols, legend = false,
267          xtickfontsize = 9)
268
269 p3 = bar(names_sorted, mse_vals,
270          xlabel = "Algorithm", ylabel = "MSE (m²)",
271          title = "Mean Squared Error",
272          color = bar_cols, legend = false,
273          xtickfontsize = 9)
274
275 p_all = plot(p1, p2, p3, layout = @layout([a; b c]), size = (1000, 900))
276 end
277
278 # 8ce2732c-e3b8-4888-9f25-7046b0782d98
279 md"""
280 Therefore, we estimate
281
282 ``X = 0.18813, \zeta=0.01896, w_n = 19.9328, =0.1525``.
283
284 where ``w_n = \sqrt{\frac{k}{m}}``.
285
286 the value of m and k cant be determined by value.
287
288 """
289
290 # 057655cf-c369-4b36-bdfe-cc77ff1a85f4
291 begin
292     const X_amp = 0.18813
293     const zeta0 = 0.01896
294     const wn    = 19.9328
295     const phi   = 0.1525
296
297     # Initial conditions from the fitted curve at t = 0
298     wd0 = sqrt(1.0 - zeta0^2) * wn
299     x0 = X_amp * cos(phi)
300     xd0 = X_amp * (-zeta0 * wn * cos(phi) + wd0 * sin(phi))
301
302     # — Analytical free-vibration response for arbitrary ζ —————
303     function free_response(t_vec, zeta, wn, x0, xd0)
304         if abs(zeta - 1.0) < 1e-6 # critically damped
305             A = x0
306             B = xd0 + wn * x0
307             return 0. (A + B * t_vec) * exp(-wn * t_vec)
308         elseif zeta < 1.0 # underdamped
309             wd = sqrt(1.0 - zeta^2) * wn
310             A = x0
311             B = (xd0 + zeta * wn * x0) / wd
312             return 0. exp(-zeta * wn * t_vec) * (A * cos(wd * t_vec) + B * sin(wd * t_vec))
313         else # overdamped
314             sq = sqrt(zeta^2 - 1.0)
315             r1 = (-zeta + sq) * wn
316             r2 = (-zeta - sq) * wn
317             A = (xd0 - r2 * x0) / (r1 - r2)
318             B = x0 - A
319             return 0. A * exp(r1 * t_vec) + B * exp(r2 * t_vec)
320         end
321     end
322
323     # — Sweep
324     ↪
325     zeta_vals = [0.0, 0.1, 0.3, 0.5, 0.7, 0.9, 1.0, 1.5, 2.0]

```

```

325     t_vec      = range(0.0, 2.0, length = 2000)  # 0-2 s captures decay well
326
327     # Color gradient: blue (undamped) → red (overdamped)
328     colors = cgrad(:RdYlBu, length(zeta_vals), rev = true)
329
330     # — Plot
331     ↪
332     p = plot(
333         xlabel = "Time (s)",
334         ylabel = "Displacement (m)",
335         title = "Free Vibration - Effect of Damping Ratio ( $\zeta = \zeta$  rad/s)",
336         legend = :topright,
337         size   = (1000, 600),
338         lw     = 1.8
339     )
340
341     for (i, zeta) in enumerate(zeta_vals)
342         x_ = free_response(collect(t_vec), zeta, wn, x0, xd0)
343
344         region = zeta < 1.0 ? "underdamped" :
345             abs(zeta - 1.0) < 1e-6 ? "critically damped" : "overdamped"
346
347         label_str = " $\zeta = \zeta$  ( $\zeta$ )"
348         plot!(p, t_vec, x_, label = label_str, color = colors[i], lw = 1.8)
349     end
350
351     # Mark the original fitted  $\zeta$ 
352     x_orig = free_response(collect(t_vec), zeta0, wn, x0, xd0)
353     plot!(p, t_vec, x_orig,
354         label = " $\zeta = \zeta$  (fitted)",
355         color  = :black,
356         lw     = 2.2,
357         linestyle = :dash)
358 end
359

```

B.3.2 week2.jl — 2주차: 가진 응답 및 질량-강성 Pareto

Rotary 엔진의 접선력을 Fourier 분해하여 날개 1DOF 운동방정식의 외력으로 사용한다. DifferentialEquations.jl 의 Tsit5 적분기로 시간 응답을 구하고, 질량-강성 2D grid에 대한 brute-force Pareto frontier를 Threads.@threads 로 병렬 계산한다. RPM sweep으로 공진 RPM 대역을 확인한 뒤 주파수 분리 조건을 통해 권장 강성 상한을 도출한다.

```

week2.jl
1  ### A Pluto.jl notebook ###
2  # v0.20.25
3
4  using Markdown
5  using InteractiveUtils
6
7  # This Pluto notebook uses @bind for interactivity. When running this notebook outside of Pluto, the following 'mock version' of
8  ↪ @bind gives bound variables a default value (instead of an error).
9  macro bind(def, element)
10     #! format: off
11     return quote
12         local iv = try Base.loaded_modules[Base.PkgId(Base.UUID("6e696c72-6542-2067-7265-42206c756150")),
13             ↪ "AbstractPlutoDingjies"].Bonds.initial_value catch; b -> missing; end
14         local el = $(esc(element))
15         global $(esc(def)) = Core.applicable(Base.get, el) ? Base.get(el) : iv(el)
16     end
17     #! format: on
18 end
19
20 # aa541eea-403b-4b2c-a199-a89d33e1bccf
21 using Plots, DifferentialEquations, LaTeXStrings, PlutoUI, FFTW, DSP, ForwardDiff
22
23 # ca4b4f26-8f5d-43b3-b681-b6bc64996ad2
24 include("../resources/week2/tangential_force.jl");
25
26 # 37f7f6d2-3ce2-4cf7-b6fa-218e95ef16f6
27 include("../resources/week2/fourier_torque.jl");
28
29 # a6bbd2e0-5989-11f1-aeb3-0925625a1384
30 md"""
31 # WEEK 2
32 """
33
34 # 35966b7b-3093-4773-95eb-5b01430b16bb
35 md"""
36 the typical single pistol engine is radial engine, as shown below
37 ![radial engine]()
38
39 since horizontal mode's natural frequency is high enough and less vibration then vertical direction, we only consider vertical
40 ↪ vibration in this case. if the motion of engine is stationary enough, neglecting moment force couple and therefore consider
41 ↪ only effect by force. For sake of simplicity, if we take equivalent term and assume the total effect by radial engine is same
42 ↪ produced by single cylinder.(at least tangential force, what we are considering most for now)then we can express total base
43 ↪ excitation by engine, as given in *Analysis of torsional vibration in internal combustion
44 engines: modelling and experimental validation*(DOI: 10.1243/14644193JMBD126)

```

```

40 , we can assume the force is like below.
41 ""
42
43 # 190721f2-a1a3-4f36-8c0d-3c5875275364
44 @bind rpm NumberField(0:2e5, default=2e3)
45
46 # 5c236fe5-2a9e-4432-92d4-281b8a7e9cd2
47 plot_rpm_fig(rpm)
48
49 # 09cc22e0-cd14-48ff-b492-655b559eee4c
50 r = fourier_decompose(rpm, 24); #return rpm based behavior on upper engine design
51
52 # ebc6bbbc7-f1eb-4c08-834f-4b1022d76a61
53 md""
54 Based on the graph above, the horizontal force component can be neglected. The vertical force component is then expressed as a sum
55 ↪ of sinusoids via Fourier decomposition (order = 24).""
56
57 # 8fb0efcb-ef8-4614-9bbf-73bdb8c9e815
58 r.plot
59
60 # a4c7e5fe-bea9-414a-9afe-490e06624578
61 md""
62 note n/2 since 4 stroke engine require 2 revolution to finish full cycle ""
63
64 # 2dd3b37e-511a-4365-8bdb-b3eb5281930b
65 md""
66 Using the Fourier-decomposed excitation ``Y(t)`` derived above and the equation of motion from Week 1,
67
68 ``m\ddot{x} + c\dot{x} + kx = \sum_{i=1}^{\infty} H(i\omega)\backslash,e^{i\omega t} = Y(t)``
69
70 we assume the wing retains the same modal properties as identified in Week 1: ``\zeta = 0.01896``, ``\omega_n = 19.9328`` rad/s.
71
72 **For simplicity, the wing mass is fixed at** ``m = 100`` **kg.**
73 ""
74
75 # 54daf206-657f-4fdb-8a70-48f99fd973da
76 @bind t_last NumberField(0:0.1:30, default=10)
77
78 # 43e20536-6326-4cac-8fea-5874280d2dbb
79 begin
80     # — Parameters
81     ↪
82     wn = 19.3828
83     m = 100
84     # mass [kg]
85     c = 0.01896*(2*m*19.9328) # damping coeff [N.s/m]
86     k = wn^2*(m) # stiffness [N/m]
87     y(t) = 0.5*evaluate_fourier(t,r.omegas,r.coefficients) #[N]
88     #! divide by half assume aircraft is fixed wing aircraft.
89
90     # — Equation of motion: m*x'' + c*x' + k*x = y(t)
91     # State: u = [x, x'] #state space representation
92     function eom!(du, u, p, t)
93         m, c, k = p
94         du[1] = u[2]
95         du[2] = (y(t) - c*u[2] - k*u[1]) / m
96     end
97
98     u0 = [0.0, 0] # initial conditions: x(0)=0, x'(0)=0
99     tspan = (0.0,t_last)
100     p = (m, c, k)
101     prob = ODEProblem(eom!, u0, tspan, p)
102     sol = solve(prob, Tsit5(), reltol=1e-8, abstol=1e-8)
103
104     # — Plot
105     ↪
106     t_vec = range(tspan[1], tspan[2], length=2000)
107     x_vec = [sol(t)[1] for t in t_vec]
108     y_vec = [y(t) for t in t_vec]
109
110     plt = plot(t_vec, y_vec, label="y(t) [N]", lw=1.5, ls=:dash, color=:orange,
111               xlabel="Time [s]", ylabel="Force [N]", legend=:topright,
112               title="Damped Forced Vibration (m=$(m), c=$(round(c)), k=$(round(k)))")
113     plt2 = twinx(plt)
114     plot!(plt2, t_vec, x_vec, label="x(t) [m]", lw=2, color=:steelblue,
115           ylabel="Displacement [m]", legend=:topleft)
116
117     #savefig(ptot, "week2_total.png")
118     end
119
120     # 6fd24617-17c7-4ecc-b1b7-a851a4b0ba49
121     begin
122         plot(t_vec,y_vec,xtlims=(0,1),title=(L"Y(t)"))
123         #savefig(p_force, "week2_only_force.png")
124     end
125
126     # 9bada243-2fc2-4712-a5da-03c32c8925db
127     begin
128         plot(t_vec,x_vec,title="displacement")
129         #savefig(p_disp, "week2_only_displacement.png")
130     end

```

```

129
130 end
131
132 # 157801b1-244f-49f0-ae7-514373a308b1
133 md"""
134 ## Wing Mass vs. Peak Amplitude - Pareto Frontier
135 """
136
137 # dc069521-aba4-4d8b-88da-65c0e0cdc89b
138 md"""
139 Assuming the natural frequency is held constant, we investigate how the peak steady-state amplitude varies with wing mass.
140 """
141
142 # 17a17e4c-7059-4da9-9cf4-996b343c005e
143 begin
144     m_range = collect(100:100:15000)
145     m_max = Vector{Float64}(undef, length(m_range))
146     Threads.@threads for i in eachindex(m_range)
147         m_i = m_range[i]
148         prob_new = remake(prob; p=(m_i, 0.01896*2*m_i*wn, m_i*wn^2))
149         sol_ = solve(prob_new, Tsit5(), reltol=1e-8, abstol=1e-8)
150         x_vec_ = [sol_(t)[1] for t in t_vec]
151         m_max[i] = maximum(abs.(x_vec_))
152     end
153 end
154
155 # faa9a60f-42c7-423d-9847-4e5491c34b0d
156 begin
157     # Brute-force 2D grid: vary mass AND stiffness independently (not tied via wn).
158     # For each (m, k) pair simulate peak amplitude, then extract Pareto-optimal front.
159     ζ_bf = 0.01896
160     m_bf_grid = collect(100:500:15000)
161     k_bf_grid = exp.(range(log(5e2), log(8e6), length=30))
162     all_combos = collect(Iterators.product(m_bf_grid, k_bf_grid))
163
164     bf_results = Vector{Tuple{Float64,Float64}}(undef, length(all_combos))
165     Threads.@threads for idx in eachindex(all_combos)
166         m_i, k_i = all_combos[idx]
167         c_i = 2ζ_bf * sqrt(m_i * k_i)
168         sol_i = solve(remake(prob; p=(m_i, c_i, k_i)), Tsit5(), reltol=1e-8, abstol=1e-8)
169         x_i = [sol_i(t)[1] for t in t_vec]
170         bf_results[idx] = (m_i, maximum(abs.(x_i)))
171     end
172
173     # A point (m_i, a_i) is Pareto-optimal if no other point (m_j, a_j) has
174     # m_j ≤ m_i AND a_j ≤ a_i with at least one strict inequality.
175     bf_pareto = filter(bf_results) do (m_i, a_i)
176         !any(bf_results) do (m_j, a_j)
177             m_j ≤ m_i && a_j ≤ a_i && (m_j < m_i || a_j < a_i)
178         end
179     end
180     sort!(bf_pareto, by = first)
181     bf_m = first.(bf_pareto)
182     bf_a = last.(bf_pareto);
183 end
184
185 # e8a2b4c1-3f5d-4e6b-8a9c-0d1e2f3a4b5c
186 begin
187     # — Constraints (engineering inputs - set by user) —————
188     x_limit = 1e-2 # max allowable amplitude [m] + adjust to your spec
189     m_min = 100.0 # structural minimum mass [kg] + adjust to your spec
190
191     # — Theoretical 1/m reference calibrated to first data point —————
192     C_ref = m_max[1] * m_range[1]
193     m_ref_line = 10 .^ range(log10(m_range[1]), log10(m_range[end]), length=300)
194
195     plt_pareto = plot(m_range, m_max,
196         xlabel="Mass [kg]", ylabel="Max Amplitude [m]",
197         xscale=:log10, yscale=:log10,
198         title="Wing Mass vs Peak Amplitude - Pareto Frontier",
199         label="Simulated frontier", lw=2, color=:steelblue,
200         marker=:circle, markersize=2, legend=:topright)
201
202     # 1/m reference
203     plot!(plt_pareto, m_ref_line, C_ref ./ m_ref_line,
204         ls=:dash, color=:gray, lw=1.5, label="1/m reference (slope = 1)")
205
206     # amplitude constraint (horizontal)
207     hline!(plt_pareto, [x_limit],
208         ls=:dot, color=:red, lw=2, label="Amplitude limit $(x_limit) m")
209
210     # mass constraint (vertical)
211     vline!(plt_pareto, [m_min],
212         ls=:dot, color=:forestgreen, lw=2, label="Min. mass $(m_min) kg")
213
214     # baseline design point
215     scatter!(plt_pareto, [m_range[1]], [m_max[1]],
216         markersize=8, color=:orange, markershape=:star5, label="Baseline (m=$(m_range[1]) kg)")
217
218     # Brute-force Pareto frontier: non-dominated designs from 2D (mass × stiffness) grid
219     plot!(plt_pareto, bf_m, bf_a,
220         lw=2, color=:crimson, ls=:dashdot,

```

```

221         marker=:diamond, markersize=4, label="Brute-force Pareto frontier")
222
223     plt_pareto
224 end
225
226 # 2be28692-a16d-4556-a39e-d4400207fde9
227 md"""
228 This indicate we need to adjust stiffness to get better system
229 """
230
231 # fa3cdf7c-9af5-4251-b4a3-bf581ad8d09c
232 md"""
233 ## Recommended Stiffness k for Flight-Envelope Safety - Engineering Justification
234 """
235
236
237 # a1b2c3d4-e5f6-4a7b-8c9d-0elf2a3b4c5d
238 md"""
239 ### What is the Flight Envelope?
240
241 In this context, the flight envelope refers to the RPM range over which the engine operates during normal flight.
242
243 Fundamental excitation frequency of a 4-stroke radial engine:  $\omega_{exc} = \frac{\text{RPM}}{60} \cdot \pi$  [rad/s]
244
245 If the wing natural frequency  $\omega_n = \sqrt{k/m}$  coincides with this excitation frequency, resonance occurs and the
    ↳ vibration amplitude grows rapidly.
246
247 Design principle - Frequency Separation (Soft Design): Choose  $k$  such that  $\omega_n < \eta \cdot \omega_{exc, min}$  holds
    ↳ across the entire flight envelope. ( $\eta = 0.8$ , 20% safety margin)
248 """
249
250 # b2c3d4e5-f6a7-4b8c-9d0e-1f2a3b4c5d6e
251 begin
252     rpm_flight = collect(600:50:2700)
253     x_peak_rpm = Vector{Float64}(undef, length(rpm_flight))
254
255     for i in eachindex(rpm_flight)
256         r_i = fourier_decompose(rpm_flight[i], 24)
257         oms_i = copy(r_i.omegas)
258         coef_i = copy(r_i.coefficients)
259         yi = let oms = oms_i, coef = coef_i
260             t -> 0.5 * evaluate_fourier(t, oms, coef)
261         end
262         prob_i = ODEProblem(
263             (du, u, p, t) -> begin
264                 m_, c_, k_ = p
265                 du[1] = u[2]
266                 du[2] = (yi(t) - c_*u[2] - k_*u[1]) / m_
267             end,
268             [0.0, 0.0], (0.0, 20.0), (m, c, k)
269         )
270         sol_i = solve(prob_i, Tsit5(), reltol=1e-6, abstol=1e-6)
271         t_ss = range(15.0, 20.0, length=500)
272         x_peak_rpm[i] = maximum(abs.([sol_i(t)[1] for t in t_ss]))
273     end
274 end
275
276 # c3d4e5f6-a7b8-4c9d-0elf-2a3b4c5d6e7f
277 begin
278     x_safe_env = 5e-2 # allowable peak amplitude [m] - adjust to your spec
279
280     plt_env = plot(rpm_flight, x_peak_rpm,
281         xlabel="Engine RPM", ylabel="Steady-state Peak Amplitude [m]",
282         title="Amplitude Response across Flight Envelope (k = $(round(Int, k)) N/m)",
283         label="Peak displacement |x(t)|", lw=2, color=:steelblue,
284         marker=:circle, markersize=3, legend=:topright)
285
286     hline!(plt_env, [x_safe_env],
287         ls=:dash, color=:red, lw=2,
288         label="Allowable amplitude limit $(x_safe_env) m")
289
290     resonant_rpm_1st = wn / (π/60)
291     vline!(plt_env, [resonant_rpm_1st],
292         ls=:dot, color=:orange, lw=2,
293         label="1st resonance RPM ≈ $(round(Int, resonant_rpm_1st))")
294
295     plt_env
296     savefig(plt_env, "week2_rpm_relative")
297 end
298
299 # d4e5f6a7-b8c9-4d0e-1f2a-3b4c5d6e7f8a
300 begin
301     rpm_min_fe = 600.0
302     rpm_max_fe = 2700.0
303     η_sep = 0.8
304
305     ω_exc_min_fe = rpm_min_fe * π / 60
306     ω_exc_max_fe = rpm_max_fe * π / 60
307     wn_limit_fe = η_sep * ω_exc_min_fe
308     k_rec = m * wn_limit_fe^2
309
310     (;

```

```

311         @_exc_min_rad_s = round(@_exc_min_fe, digits=2),
312         @_exc_max_rad_s = round(@_exc_max_fe, digits=2),
313         wn_limit_rad_s   = round(wn_limit_fe, digits=2),
314         k_recommended    = round(Int, k_rec),
315         k_current         = round(Int, k),
316         wn_current_rad_s  = round(wn, digits=4),
317         is_safe           = k <= k_rec
318     );
319 end
320
321 # abfacf7b-e534-43d4-90ef-d008228a9e44
322 md"""
323 ### Engineering Justification
324
325 **1. Flight Envelope Definition**
326 Operating RPM range for a small piston-engine aircraft: 600 - 2700 RPM
327 → Fundamental excitation frequency range: $(round(600*\pi/60, digits=1)) - $(round(2700*\pi/60, digits=1)) rad/s
328
329
330 **2. Resonance Avoidance Condition (Frequency Separation)**
331 ``\omega_n < 0.8 \times \omega_{exc,min} = 0.8 \times`` $(round(@_exc_min_fe, digits=1)) ``\approx`` $(round(wn_limit_fe,
332   ↳ digits=1)) rad/s
333 → Recommended stiffness upper bound: ``k \leq m \cdot \omega_{n,lim}^2`` = $(round(Int, k_rec)) N/m
334
335 **3. Current Design Verification**
336 $\omega_n$ = $(round(wn, digits=2))rad/s $(k <= k_rec ? "<" : ">") $(round(wn_limit_fe, digits=1))rad/s &&
337 k = $(round(Int, k)) N/m $(k <= k_rec ? "<" : ">") $(round(Int, k_rec)) N/m
338
339 → $(k <= k_rec ? "** Safe: no resonance occurs across the entire flight envelope**" : "** Unsafe: resonance possible - stiffness
340   ↳ must be reduced**")
341
342 **4. Conclusion**
343 Keeping wing stiffness at k ≤ $(round(Int, k_rec))N/m
344
345 ensures no resonance occurs anywhere in the flight envelope (600 - 2700 RPM). The current design k $\approx$ $(round(Int, k)) N/m
346   ↳ satisfies this criterion.
347
348 """

```

B.3.3 tangential_force.jl — 크랭크핀 접선력 모델

단일 실린더 내연기관의 크랭크핀 접선력을 계산하는 라이브러리. Mendes et al.(2008) 논문의 식 (8)–(13)을 구현 하며, 가스 하중(F_{tg})과 왕복 관성력(F_{ta})의 합으로 결과 접선력 F_t 를 산출한다. `sweep_cycle()`은 1회 4행정 사이클(0–720도)에 걸쳐 모든 성분을 벡터로 반환하고, `plot_rpm_fig()`는 임의 RPM에서의 접선력 그래프를 생성해 `week2.jl`의 인터랙티브 슬라이더에 연결된다.

```

----- week2/tangential_force.jl -----
1 # -----
2 # tangential_force.jl
3 #
4 # Crankpin tangential force and excitation moment for a single cylinder of
5 # an internal-combustion engine, as a function of crank angle alpha and
6 # angular velocity omega.
7 #
8 # Reference:
9 # Mendes A.S., Meirelles P.S., Zampieri D.E. (2008),
10 # "Analysis of torsional vibration in internal combustion engines:
11 # modelling and experimental validation",
12 # Proc. IMechE Vol. 222 Part K (J. Multi-body Dynamics), pp. 155-178.
13 #
14 # Implements equations (8)-(13). Target reproduction: Fig. 14
15 # (crankpin tangential forces at 2000 and 2550 r/min).
16 #
17 # Pressure model: parametric Gaussian approximation of Figs. 18 & 19.
18 # The paper uses MEASURED p(alpha) curves; this is a stand-in only and
19 # must be replaced with measured data for quantitative agreement.
20 # -----
21
22 using Plots
23
24 # == Engine geometry & oscillating mass (paper p.156) =====
25 const D_PISTON = 0.105      # piston diameter dp [m]
26 const STROKE   = 0.137      # piston stroke s [m]
27 const R_CRANK  = STROKE / 2 # crank radius r [m] (s = 2r)
28 const L_ROD    = 0.207      # con-rod length L [m]
29 const M_OSC    = 2.521      # oscillating mass ma (piston + mab) [kg]
30 const LAMBDA   = R_CRANK / L_ROD # lambda = r/L ~ 0.331
31
32 rpm_to_omega(n_e_rpm) = 2pi * n_e_rpm / 60.0 # [rad/s]
33
34 # == Kinematics =====
35 "Connecting-rod inclination beta from sin(beta) = lambda * sin(alpha)."
36 beta_angle(alpha) = asin(LAMBDA * sin(alpha))
37
38 "Geometric factor sin(alpha+beta)/cos(beta), shared by eqs (9) and (11)."

```

```

39 function geo_factor(alpha)
40     b = beta_angle(alpha)
41     return sin(alpha + b) / cos(b)
42 end
43
44 # === Cylinder pressure (parametric Gaussian approximation) =====
45 # Peak combustion pressure read off Fig. 19 of the paper [Pa]
46 function p_peak_fig19(n_e_rpm)
47     rpm = [1000.0, 1400.0, 1600.0, 1800.0, 2000.0, 2200.0, 2400.0, 2550.0]
48     pp = [14.2, 14.5, 14.5, 14.8, 15.0, 15.0, 13.0, 6.0] .* 1e6
49     n = clamp(n_e_rpm, rpm[1], rpm[end])
50     i = something(findlast(<=(n), rpm), 1)
51     i = min(i, length(rpm) - 1)
52     t = (n - rpm[i]) / (rpm[i+1] - rpm[i])
53     return pp[i] + t * (pp[i+1] - pp[i])
54 end
55
56 """
57     cylinder_pressure(alpha_rad, n_e_rpm; p_atm, alpha_peak_deg, sigma_deg) -> Pa
58
59     Approximate cylinder pressure for one 4-stroke cycle (0..720 deg).
60     Centre of combustion ~12 deg ATDC (compression TDC = 360 deg by convention
61     used in Fig. 18). Replace by measured p(alpha) for accurate analyses.
62 """
63 function cylinder_pressure(alpha_rad, n_e_rpm;
64                             p_atm=1.0e5,
65                             alpha_peak_deg=372.0,
66                             sigma_deg=25.0)
67     alpha_deg = mod(rad2deg(alpha_rad), 720.0)
68     p_peak = p_peak_fig19(n_e_rpm)
69     return p_atm + (p_peak - p_atm) *
70         exp(-0.5 * ((alpha_deg - alpha_peak_deg) / sigma_deg)^2)
71 end
72
73 # === Forces (eqs 8 - 12) =====
74 "Gas load on piston, eq. (8): F_g = pi * dp^2 / 4 * p."
75 gas_load(p) = pi * D_PISTON^2 / 4 * p
76
77 "Tangential gas load, eq. (9): F_tg = F_g * sin(alpha+beta)/cos(beta)."

```

```

131 # alpha from TDC (cylinder axis), rotation increases alpha.
132 # crankpin position : (r sin a, r cos a)
133 # tangent unit vector : (cos a, -sin a) (direction of rotation)
134 F_t_x = F_t .* cos(a_rad) # horizontal component (sideways)
135 F_t_y = -F_t .* sin(a_rad) # vertical component (along cylinder axis, + up)
136 return (alpha_deg = collect(alpha_deg),
137         F_tg = F_tg, F_ta = F_ta, F_t = F_t,
138         F_t_x = F_t_x, F_t_y = F_t_y,
139         M_t = moment_from_Ft.(F_t))
140 end
141
142 # === Plot reproducing Fig. 14 =====
143 function plot_fig14(; savepath::Union{Nothing,String}=nothing)
144     r1 = sweep_cycle(2000)
145     r2 = sweep_cycle(2550)
146
147     common = (xlabel="Crankshaft angle [deg]", ylabel="[N]",
148              xticks=0:60:720, legend=:topleft, lw=1.5,
149              framestyle=:box)
150
151     p1 = plot(r1.alpha_deg, r1.F_tg, label="Tangential gas load",
152              color=:gray, title="Tangential Forces at 2000 rpm"; common...)
153     plot!(p1, r1.alpha_deg, r1.F_ta, label="Tangential inertia force",
154           color=:black)
155     plot!(p1, r1.alpha_deg, r1.F_t, label="Resultant tangential force",
156           color=:black, ls=:dash)
157
158     p2 = plot(r2.alpha_deg, r2.F_tg, label="Tangential gas load",
159              color=:gray, title="Tangential Forces at 2550 rpm"; common...)
160     plot!(p2, r2.alpha_deg, r2.F_ta, label="Tangential inertia force",
161           color=:black)
162     plot!(p2, r2.alpha_deg, r2.F_t, label="Resultant tangential force",
163           color=:black, ls=:dash)
164
165     fig = plot(p1, p2, layout=(2,1), size=(900, 800))
166     savepath != nothing && savefig(fig, savepath)
167     return fig
168 end
169
170 # === plot for interactive query of tangential forces at a single speed =====
171 function plot_rpm_fig(speed_1; savepath::Union{Nothing,String}=nothing)
172     r1 = sweep_cycle(speed_1)
173
174     common = (xlabel="Crankshaft angle [deg]", ylabel="[N]",
175              xticks=0:60:720, legend=:topleft, lw=1.5,
176              framestyle=:box)
177
178     p1 = plot(r1.alpha_deg, r1.F_tg, label="Tangential gas load",
179              color=:gray, title="Tangential Forces at 2000 rpm"; common...)
180     plot!(p1, r1.alpha_deg, r1.F_ta, label="Tangential inertia force",
181           color=:black)
182     plot!(p1, r1.alpha_deg, r1.F_t, label="Resultant tangential force",
183           color=:black, ls=:dash)
184
185     fig = plot(p1, size=(900, 800))
186     #savepath != nothing && savefig(fig, savepath)
187     return fig
188 end
189
190 # Run with:
191 # julia> include("tangential_force.jl")
192 # julia> plot_fig14(savepath="imgs/tangential_force_fig14.png")
193 #
194 # Single-point query:
195 # julia> tangential_force(deg2rad(380.0), rpm_to_omega(2000),
196 #                        cylinder_pressure(deg2rad(380.0), 2000))
197 #=If my sign convention for F_t,y (positive up) is the opposite of what you wanted,
198 just flip the sign in line 132 of tangential_force.jl (F_t_y = -F_t .* sin.(a_rad) + F_t .* sin.(a_rad)).=#

```

B.3.4 fourier_torque.jl — 점선력의 Fourier 분해

tangential_force.jl에서 얻은 F_t 파형을 Fourier 급수로 분해하는 라이브러리. fourier_decompose()가 DFT로 복소 계수 H_n 과 각주파수 ω_n 을 추출하고, evaluate_fourier()로 임의의 시간 t 에서 재건 파형을 계산한다. 내부 _stfft_spectrogram() 함수는 3주기 분을 타일링하여 STFT 스펙트로그램을 생성, 시간-주파수 에너지 분포를 시각화한다.

```

----- week2/fourier_torque.jl -----
1 # -----
2 # fourier_torque.jl
3 #
4 # Fourier-series decomposition of the resultant crankpin tangential force
5 # F_t (and equivalently the per-cylinder excitation torque M_t = F_t * r)
6 # obtained from `sweep_cycle` in `tangential_force.jl`.
7 #
8 # Signal period: one full 4-stroke cycle = 720 deg crank angle = 2 engine
9 #                revolutions, so the fundamental Fourier frequency is
10 #                omega_fund = omega_engine / 2.
11 # This is the same convention as eq. (17) of Mendes et al. (2008):
12 #

```

```

13 # M_t(t) = A0/2 + sum_{n=1..N} [ C_n exp(+i n omega_fund t)
14 #                               + conj(C_n) exp(-i n omega_fund t) ]
15 #
16 # Two-sided complex form (returned by this module):
17 #
18 # F_t(t) = sum_{n = -N..N} H_n * exp( i * omega_n * t )
19 #
20 # with omega_n = n * omega_fund and H_{-n} = conj(H_n).
21 # -----
22
23 #include("tangential_force.jl")
24
25 using FFTW
26 using Plots
27
28 # --- internal: STFT spectrogram of a periodic signal -----
29 # Tiles the one-cycle signal `n_cycles` times so the sliding window has room.
30 # Returns (magnitude, crank_angle_deg_axis, engine_order_axis).
31 function _stft_spectrogram(sig, n_per_cycle;
32                             n_cycles::Integer = 3,
33                             win_frac::Real = 0.5,
34                             hop_frac::Real = 0.025)
35     tiled = repeat(sig, n_cycles)
36     N = length(tiled)
37     win_len = max(8, round(Int, n_per_cycle * win_frac))
38     hop = max(1, round(Int, n_per_cycle * hop_frac))
39     w = @. 0.5 * (1 - cos(2pi * (0:win_len-1) / max(win_len - 1, 1)))
40     n_win = (N - win_len) ÷ hop + 1
41     nbins = win_len ÷ 2 + 1
42     spec = zeros(nbins, n_win)
43     for k in 0:n_win-1
44         s = k * hop + 1
45         seg = view(tiled, s:s+win_len-1) .* w
46         spec[:, k+1] = abs.(fft(seg))[1:nbins]
47     end
48     crank_deg = ((0:n_win-1) .* hop .+ (win_len - 1)/2) .* (720.0 / n_per_cycle)
49     orders = (0:win_len-2) .* (n_per_cycle / (2 * win_len)) # engine orders
50     return spec, crank_deg, orders
51 end
52
53 """
54     fourier_decompose(rpm, order; signal=:F_t_y, n_samples=1440,
55                       max_order_display=15) -> NamedTuple
56
57 Fourier-decompose one of the per-cycle force/moment traces from
58 `sweep_cycle(rpm)`. Valid values for `signal`:
59 :F_t_y -- vertical component of the tangential force (default)
60 :F_t_x -- horizontal component of the tangential force
61 :F_t   -- scalar tangential force (magnitude along tangent)
62 :M_t   -- excitation moment (= F_t * r)
63
64 The cycle is sampled at `n_samples` uniform crank-angle points over
65 0..720 deg (excluding the duplicate endpoint), the DFT is computed, and
66 the first `order` harmonics (plus the DC term) are kept.
67
68 The returned `plot` field is a 3-panel figure:
69 1. Top -- original signal vs Fourier reconstruction (time domain).
70 2. Middle -- one-sided amplitude spectrum |H_n| vs engine order.
71 3. Bottom -- STFT spectrogram in dB (signal tiled 3 cycles).
72
73 Returned fields
74 omegas      :: Vector{Float64}    two-sided frequencies omega_n [rad/s]
75                                     length = 2*order + 1, ordered -N..N
76 coefficients :: Vector{ComplexF64} Fourier coefficients H_n matching omegas
77 omega_fund   :: Float64           fundamental = omega_engine / 2 [rad/s]
78 alpha_deg   :: Vector{Float64}    sample crank angles [deg]
79 signal      :: Vector{Float64}    original sampled signal [N or N*m]
80 reconstructed :: Vector{Float64}  Fourier reconstruction at same samples
81 spectrogram :: NamedTuple         (magnitude, crank_deg, orders) from STFT
82 plot        :: Plots.Plot         3-panel composite figure
83
84 Reconstruction at arbitrary time `t` [s]:
85 f_t = real(sum(coefficients .* exp.(1im .* omegas .* t)))
86 or use the helper `evaluate_fourier(t, omegas, coefficients)`.
87 """
88 function fourier_decompose(rpm::Real, order::Integer;
89                             signal::Symbol = :F_t_y,
90                             n_samples::Integer = 1440,
91                             max_order_display::Real = 15.0)
92     # --- sample one full cycle -----
93     alpha_deg = collect(range(0.0, 720.0, length = n_samples + 1))[1:end-1]
94     res = sweep_cycle(rpm; alpha_deg = alpha_deg)
95     sig = signal === :M_t ? res.M_t :
96           signal === :F_t_x ? res.F_t_x :
97           signal === :F_t_y ? res.F_t_y :
98           signal === :F_t ? res.F_t :
99           error("fourier_decompose: unknown signal $(signal); use :F_t_y, :F_t_x, :F_t, or :M_t")
100     N = length(sig)
101
102     omega_engine = rpm_to_omega(rpm) # [rad/s]
103     omega_fund = omega_engine / 2 # 720 deg period -> half engine speed

```

```

105 # --- DFT --> complex Fourier coefficients -----
106 Y = fft(sig)
107 c = Y ./ N                                     # c_n, n = 0,1,...,N-1
108                                           # c_{N-n} = conj(c_n) for real sig
109
110 nmax = min(order, N ÷ 2)
111 c_pos = c[1:nmax+1]                           # n = 0, 1, ..., nmax
112
113 # two-sided arrays, ordered -nmax .. nmax
114 omegas = collect(-nmax:nmax) .* omega_fund
115 coeffs = ComplexF64[ n >= 0 ? c_pos[n + 1] : conj(c_pos[-n + 1])
116                      for n in -nmax:nmax ]
117
118 # --- reconstruct on the same crank-angle grid -----
119 t = deg2rad(alpha_deg) ./ omega_engine # time [s] at each sample
120 recon = [ real(sum(coeffs .* exp.(1im .* omegas .* tt))) for tt in t ]
121
122 # --- STFT spectrogram (tile 3 cycles) -----
123 spec, sg_crank_deg, sg_orders = _stft_spectrogram(sig, N; n_cycles = 3)
124 spec_db = 20 .* log10.(spec ./ maximum(spec) .+ 1e-6)
125 keep = findall(sg_orders .<= max_order_display)
126
127 # --- plot panels -----
128 ylab = signal === :M_t ? "Excitation moment M_t [N m]" :
129       signal === :F_t_x ? "F_t horizontal component F_t,x [N]" :
130       signal === :F_t_y ? "F_t vertical component F_t,y [N]" :
131       "Resultant tangential force F_t [N]"
132
133 lmarg = 12Plots.mm
134
135 t_ms = t .* 1e3                                # time axis in milliseconds
136 ms_per_deg = 60_000.0 / (rpm * 360)           # 1° crank in ms
137
138 fmt(x) = isinteger(x) ? string(Int(x)) :
139         string(round(x, digits = 1))
140
141 # tick positions chosen on the crank-angle grid (multiples of 60°) and
142 # labelled with both ms and deg so the user reads the same tick two ways.
143 tick_deg_time = 0:60:720
144 tick_ms_time = collect(tick_deg_time) .* ms_per_deg
145 tick_labels_t = ["$(fmt(ms)) ms\n$(d)°"
146                 for (ms, d) in zip(tick_ms_time, tick_deg_time)]
147
148 p_time = plot(t_ms, sig,
149              label = "Original", color = :black, lw = 1.5,
150              xlabel = "Time [ms] / Crank angle [deg]", ylabel = ylab,
151              xlims = (0, t_ms[end]),
152              xticks = (tick_ms_time, tick_labels_t),
153              framestyle = :box,
154              left_margin = lmarg,
155              bottom_margin = 4Plots.mm,
156              title = "Fourier reconstruction @ $(rpm) rpm, order = $(nmax)")
157 plot!(p_time, t_ms, recon,
158       label = "Fourier (n = 0..$(nmax))",
159       color = :red, lw = 1.5, ls = :dash)
160
161 # one-sided amplitude spectrum (engine orders 0, 0.5, 1, ...)
162 orders_x = collect(0:nmax) ./ 2
163 mags = Float64[ n == 0 ? abs(c_pos[1]) : 2 * abs(c_pos[n + 1])
164                for n in 0:nmax ]
165 p_spec = bar(orders_x, mags,
166             label = "|H_n|",
167             xlabel = "Engine order (n / 2)", ylabel = "Amplitude",
168             color = :steelblue, lw = 0, bar_width = 0.35,
169             xticks = 0:1:max(1, ceil(orders_x[end])),
170             framestyle = :box,
171             left_margin = lmarg,
172             title = "One-sided amplitude spectrum")
173
174 # dual tick labels for the spectrogram (x is crank angle, also show ms)
175 sg_lo = first(sg_crank_deg); sg_hi = last(sg_crank_deg)
176 tick_deg_sg = filter(d -> sg_lo <= d <= sg_hi, 0:360:2880)
177 tick_labels_sg = ["$(Int(d))°\n$(fmt(d * ms_per_deg)) ms" for d in tick_deg_sg]
178
179 p_sg = heatmap(sg_crank_deg, sg_orders[keep], spec_db[keep, :],
180              xlabel = "Crank angle [deg] / Time [ms] (3 cycles tiled)",
181              ylabel = "Engine order",
182              color = :viridis, clim = (-60.0, 0.0),
183              xticks = (tick_deg_sg, tick_labels_sg),
184              colorbar_title = "dB (rel. max)",
185              framestyle = :box,
186              left_margin = lmarg,
187              bottom_margin = 4Plots.mm,
188              title = "STFT spectrogram")
189
190 plt = plot(p_time, p_spec, p_sg, layout = (3, 1), size = (960, 1100))
191
192 return (omegas = omegas,
193        coefficients = coeffs,
194        omega_fund = omega_fund,
195        alpha_deg = alpha_deg,
196        signal = sig,

```

```

197         reconstructed = recon,
198         spectrogram = (magnitude = spec,
199                        crank_deg = sg_crank_deg,
200                        orders = sg_orders),
201         plot = plt)
202     end
203
204     """
205     evaluate_fourier(t, omegas, coefficients) -> Float64
206
207     Evaluate f(t) = sum_n H_n * exp(i * omega_n * t) at a single time `t [s]`,
208     returning the real part (the imaginary part is numerical noise for the
209     Hermitian-symmetric two-sided arrays returned by `fourier_decompose`).
210     """
211     evaluate_fourier(t, omegas, coefficients) =
212         real(sum(coefficients .* exp.(im .* omegas .* t)))
213
214     """
215     spectrum(result) -> (orders, magnitudes)
216
217     Convenience: convert a `fourier_decompose` result into a single-sided
218     amplitude spectrum (engine orders vs |2 H_n| for n >= 1, |H_0| for n = 0).
219     `orders[n+1] = n / 2` because omega_fund = omega_engine / 2.
220     """
221     function spectrum(result)
222         nmax = (length(result.omegas) - 1) ÷ 2
223         orders = collect(0:nmax) ./ 2 # engine orders
224         mags = Float64[ n == 0 ? abs(result.coefficients[nmax + 1]) :
225                          2 * abs(result.coefficients[nmax + 1 + n])
226                        for n in 0:nmax ]
227         return orders, mags
228     end
229
230     # --- example -----
231     # include("fourier_torque.jl")
232     # r = fourier_decompose(2000, 24) # 24 harmonics, like the paper
233     # display(r.plot)
234     # savefig(r.plot, "imgs/fourier_torque_2000rpm.png")
235     # evaluate_fourier(0.005, r.omegas, r.coefficients) # F_t at t = 5 ms
236     # orders, mags = spectrum(r)
237     # fourier_decompose(2000, 24) # vertical component (default)
238     # fourier_decompose(2000, 24; signal=:F_t_x) # horizontal component
239     # fourier_decompose(2000, 24; signal=:F_t) # scalar tangential
240     # fourier_decompose(2000, 24; signal=:M_t) # excitation moment

```

B.4 3주차: 날개의 로우패스 필터 특성과 공명현상 (MATLAB)

2DOF 날개 모델의 주파수 응답을 분석한다. Bode 선도, 임펄스 응답, Schroeder 위상 멀티사인을 이용하여 날개가 저주파 대역에서 큰 이득을 가지는 로우패스 필터 특성을 확인하고 공진 대역폭을 정량적으로 파악한다.

B.4.1 bode_response.m

wing_2dof.mat의 상태공간 모델로부터 주파수 전달함수를 구성하고, 0.1 Hz–50 Hz 로그 스윙에서 플런지 h 와 피치 θ 각각의 Bode 선도를 출력한다.

```

1 % bode_response.m -- Bode plot: plunge h and pitch theta vs gust w_g
2
3 here = fileparts(mfilename('fullpath'));
4 load(fullfile(here, 'wing_2dof.mat'), 'A', 'B', 'C_out', 'D_out', 'fn_struct');
5
6 sys = ss(A, B, C_out, D_out);
7 sys.OutputName = {'h', 'theta'};
8 sys.InputName = {'w_g'};
9
10 % frequency range spanning both structural modes
11 f = logspace(-1, 1.7, 600); % 0.1 - 50 Hz
12 om = 2*pi*f;
13
14 [mag, phs] = bode(sys, om);
15 mag_h = squeeze(mag(1,1,:)); phs_h = squeeze(phs(1,1,:));
16 mag_th = squeeze(mag(2,1,:)); phs_th = squeeze(phs(2,1,:));
17
18 fh = fn_struct(1); ft = fn_struct(2);
19
20 col_h = [0.85 0.33 0.1];
21 col_th = [0.2 0.4 0.8];
22
23 figure('Name', 'Bode Response', 'Position', [100 100 760 540]);
24
25 subplot(2,1,1)
26 semilogy(f, mag_h, 'Color', col_h, 'LineWidth', 1.8, 'DisplayName', 'h (plunge)')
27 hold on
28 semilogy(f, mag_th, 'Color', col_th, 'LineWidth', 1.8, 'DisplayName', '\theta (pitch)')
29 xline(fh, '--', 'Color', [.5 .5 .5], 'HandleVisibility', 'off')
30 xline(ft, '--', 'Color', [.5 .5 .5], 'HandleVisibility', 'off')
31 set(gca, 'XScale', 'log')
32 ylabel('Magnitude [m/(m/s)], [rad/(m/s)]')

```

```

33 title('Bode Magnitude'); legend('Location','northwest'); grid on
34
35 subplot(2,1,2)
36 plot(f, phs_h, 'Color',col_h, 'LineWidth',1.8, 'DisplayName','h')
37 hold on
38 plot(f, phs_th, 'Color',col_th, 'LineWidth',1.8, 'DisplayName','\theta')
39 xline(fh,'--','Color',[.5 .5 .5],'HandleVisibility','off')
40 xline(ft,'--','Color',[.5 .5 .5],'HandleVisibility','off')
41 set(gca,'XScale','log')
42 xlabel('Frequency [Hz]'); ylabel('Phase [deg]')
43 title('Bode Phase'); legend('Location','southwest');
44 grid on;
45 %exportgraphics(gcf, 'week3_bode_response.png', 'Resolution', 300)

```

Listing 3: bode_response.m — Bode 선도

B.4.2 impulse_response.m

상태공간 시스템에 단위 임펄스 돌풍을 가했을 때 플런지와 피치의 자유 응답을 계산한다. 응답 진폭으로부터 고유진동수와 감쇠비를 정성적으로 확인할 수 있다.

```

1 % impulse_response.m -- Impulse response: plunge h and pitch theta
2
3 here = fileparts(mfilename('fullpath'));
4 load(fullfile(here,'wing_2dof.mat'),'A','B','C_out','D_out','fn_struct');
5
6 sys = ss(A, B, C_out, D_out);
7 sys.OutputName = {'h','theta'};
8 sys.InputName = {'w_g'};
9
10 % time vector: long enough for transient to decay
11 t_end = 8;
12 opt = RespConfig;
13 opt.Amplitude = 100;
14 [y, t] = step(sys,t_end,opt); % y: [Nt x 2 x 1]
15 %support all type of inputs: step, impulse, arbitrary time series, etc.
16
17 h_imp = y(:,1) * 1000; % [mm·s] (impulse has units output·s)
18 th_imp = rad2deg(y(:,2)); % [deg·s]
19
20 fh = fn_struct(1); ft = fn_struct(2);
21
22 col_h = [0.85 0.33 0.1];
23 col_th = [0.2 0.4 0.8];
24
25 figure('Name','Impulse Response','Position',[100 100 760 420]);
26
27 subplot(2,1,1)
28 plot(t, h_imp, 'Color',col_h, 'LineWidth',1.5)
29 yline(0,'Color',[.7 .7 .7])
30 xlabel('t [s]'); ylabel('h [mm\cdots]')
31 title(sprintf('Plunge (f_h = %.2f Hz, f_\theta = %.2f Hz)', fh, ft))
32 grid on
33
34 subplot(2,1,2)
35 plot(t, th_imp, 'Color',col_th, 'LineWidth',1.5)
36 yline(0,'Color',[.7 .7 .7])
37 xlabel('t [s]'); ylabel('\theta [deg\cdots]')
38 title('Pitch')
39 grid on
40 exportgraphics(gcf, 'week3_impulse_response.png', 'Resolution', 300)

```

Listing 4: impulse_response.m — 임펄스 응답

B.4.3 multisine_function_generator.m

Schroeder 위상 멀티사인 신호를 생성하는 함수. 기본 주파수 f_0 , 고조파 수 K , RMS 진폭, 샘플링률, 주기 수를 인자로 받아 timeseries 객체로 반환한다.

```

1 function wg_ts = multisine_function_generator(f0, K, rms_amp, fs, nper)
2 % MULTISINE_FUNCTION_GENERATOR Generate a Schroeder-phase multisine as timeseries.
3 %
4 % wg_ts = multisine_function_generator() -- default params
5 % wg_ts = multisine_function_generator(f0,K,rms,fs,nper)
6 %
7 % f0      fundamental frequency [Hz]      default 0.25
8 % K       number of harmonics             default 80
9 % rms_amp desired RMS amplitude [m/s]     default 1.5
10 % fs      sample rate [Hz]               default 400
11 % nper    number of periods              default 8
12
13 if nargin < 1, f0      = 0.25; end
14 if nargin < 2, K       = 80;  end
15 if nargin < 3, rms_amp = 1.5; end
16 if nargin < 4, fs      = 400; end

```

```

17 if nargin < 5, nper = 8; end
18
19 kk = 1:K;
20 Ak = rms_amp / sqrt(K/2) * ones(1, K); % flat amplitude, RMS = rms_amp
21 phi_k = -pi * kk .* (kk-1) / K; % Schroeder phases (low crest factor)
22
23 dt = 1/fs;
24 N = round(fs * (1/f0) * nper);
25 t = (0:N-1)' * dt;
26 wg = sum(Ak .* sin(2*pi*kk.*t + phi_k), 2);
27
28 wg_ts = timeseries(wg, t, 'Name','w_g');
29 wg_ts.TimeInfo.Units = 's';
30 end

```

Listing 5: multisine_function_generator.m — Schroeder 멀티사인 생성

B.4.4 multisine_analysis.m

멀티사인 돌풍 신호에 대한 시간 응답과 Bode 선도를 2×3 서브플롯으로 한 화면에 출력한다. 상단 행은 $w_g(t)$, $h(t)$, $\theta(t)$ 시계열, 하단 행은 $|H_h(f)|$, $|H_\theta(f)|$, 위상을 나타낸다.

```

1 % multisine_analysis.m -- multisine time response + bode (2x3 figure)
2 %
3 % Row 1: time traces -- w_g(t) | h(t) [mm] | theta(t) [deg]
4 % Row 2: bode -- |H_h(f)| | |H_theta(f)| | phase (both)
5 %! ALL IN ONE: multisine time response + bode (2x3 figure)
6
7 here = fileparts(mfilename('fullpath'));
8 load(fullfile(here,'wing_2dof.mat'), 'A','B','C_out','D_out','fn_struct');
9
10 sys = ss(A, B, C_out, D_out);
11
12 % ----- input signal -----
13 wg_ts = multisine_function_generator();
14 t = wg_ts.Time;
15 wg = wg_ts.Data;
16
17 % ----- simulate -- lsim accepts continuous sys directly -----
18 y_out = lsim(sys, wg, t);
19 h_mm = y_out(:,1) * 1000;
20 th_deg = rad2deg(y_out(:,2));
21
22 fprintf('Plunge RMS = %.3f mm\n', rms(h_mm));
23 fprintf('Pitch RMS = %.4f deg\n', rms(th_deg));
24
25 % ----- bode curves -----
26 f = logspace(-1, 1.7, 600);
27 om = 2*pi*f;
28 [mag, phs] = bode(sys, om);
29 mag_h = squeeze(mag(1,1,:)); phs_h = squeeze(phs(1,1,:));
30 mag_th = squeeze(mag(2,1,:)); phs_th = squeeze(phs(2,1,:));
31
32 fh = fn_struct(1); ft = fn_struct(2);
33
34 % ----- zoom to last 2 periods for time plots -----
35 Tper = 1/0.25;
36 i_z = t >= t(end) - 2*Tper;
37 tz = t(i_z);
38
39 col_h = [0.85 0.33 0.1];
40 col_th = [0.2 0.4 0.8];
41 col_in = [0.2 0.4 0.8];
42
43 % ----- 2x3 figure -----
44 figure('Name','Multisine Analysis','Position',[80 80 1200 620]);
45
46 % --- row 1: time -----
47 subplot(2,3,1)
48 plot(tz, wg(i_z), 'Color',col_in, 'LineWidth',0.8)
49 xlabel('t [s]'); ylabel('w_g [m/s]'); title('INPUT: gust'); grid on
50
51 subplot(2,3,2)
52 plot(tz, h_mm(i_z), 'Color',col_h, 'LineWidth',1.2)
53 xlabel('t [s]'); ylabel('h [mm]'); title('TIME: plunge'); grid on
54
55 subplot(2,3,3)
56 plot(tz, th_deg(i_z), 'Color',col_th, 'LineWidth',1.2)
57 xlabel('t [s]'); ylabel('theta [deg]'); title('TIME: pitch'); grid on
58
59 % --- row 2: bode -----
60 subplot(2,3,4)
61 semilogy(f, mag_h, 'Color',col_h, 'LineWidth',1.8)
62 xline(fh, '--', 'Color',[.6 .6 .6]); xline(ft, '--', 'Color',[.6 .6 .6])
63 set(gca, 'XScale','log')
64 xlabel('f [Hz]'); ylabel('|H_h| [m/(m/s)]'); title('BODE: plunge mag'); grid on
65
66 subplot(2,3,5)
67 semilogy(f, mag_th, 'Color',col_th, 'LineWidth',1.8)

```

```

68 xline(fh, '--', 'Color', [.6 .6 .6]); xline(ft, '--', 'Color', [.6 .6 .6])
69 set(gca, 'XScale', 'log')
70 xlabel('f [Hz]'); ylabel('|H_\theta| [rad/(m/s)]'); title('BODE: pitch mag'); grid on
71
72 subplot(2,3,6)
73 plot(f, phs_h, 'Color', col_h, 'LineWidth', 1.8, 'DisplayName', 'h')
74 hold on
75 plot(f, phs_th, 'Color', col_th, 'LineWidth', 1.8, 'DisplayName', '\theta')
76 xline(fh, '--', 'Color', [.6 .6 .6], 'HandleVisibility', 'off')
77 xline(ft, '--', 'Color', [.6 .6 .6], 'HandleVisibility', 'off')
78 set(gca, 'XScale', 'log')
79 xlabel('f [Hz]'); ylabel('Phase [deg]'); title('BODE: phase');
80 legend('Location', 'southwest'); grid on
81 exportgraphics(gcf, 'week3_multisine_response.png', 'Resolution', 300)

```

Listing 6: multisine_analysis.m — 멀티사인 응답·Bode 통합 분석

B.4.5 multisine_toolbox_response.m

MATLAB Control System Toolbox의 `lsim`을 직접 사용하여 멀티사인 돌풍에 대한 응답을 검증한다.

```

1 % multisine_toolbox_response.m
2
3 here = fileparts(mfilename('fullpath'));
4 load(fullfile(here, 'wing_2dof.mat'), 'A', 'B', 'C_out', 'D_out', 'fn_struct');
5
6 sys = ss(A, B, C_out, D_out);
7
8 % ----- multisine design -----
9 f0 = 0.25; kk = 1:80; fk = kk * f0; K = length(kk);
10 Ak = 1.5 / sqrt(K/2) * ones(1, K); % flat, RMS = 1.5 m/s
11 phi_k = -pi * kk .* (kk-1) / K; % Schroeder phases
12
13 fs = 400; dt = 1/fs; Tper = 1/f0; nper = 8;
14 N = round(fs * Tper * nper);
15 t = (0:N-1)' * dt;
16 wg = sum(Ak .* sin(2*pi*fk.*t + phi_k), 2);
17
18 % ----- simulate with lsim (ZOH) -----
19 sysd = c2d(sys, dt, 'zoh');
20 y_out = lsim(sysd, wg, t); % [N x 2]: h, theta
21
22 fprintf('Plunge RMS = %.3f mm\n', rms(y_out(:,1))*1000);
23 fprintf('Pitch RMS = %.4f deg\n', rad2deg(rms(y_out(:,2))));
24
25 % ----- bode: continuous curve + values at excited lines -----
26 fc = logspace(-1, 1.5, 600);
27 [Mc, ~] = bode(sys, 2*pi*fc); % continuous analytical curve
28 [Mk, ~] = bode(sys, 2*pi*fk); % at each multisine line
29
30 Hh_c = squeeze(Mc(1,1,:)); % plunge channel, continuous
31 Hh_k = squeeze(Mk(1,1,:)); % plunge channel, at fk [1xK]
32
33 % ----- amplitudes directly from bode + design -----
34 Yamp = Hh_k .* Ak * 1000; % output amplitude [mm] = |H|*|U|
35
36 % ----- natural frequencies -----
37 fh_nat = fn_struct(1);
38 ft_nat = fn_struct(2);
39
40 % ----- 3-panel figure -----
41 figure('Name', 'Multisine FRF (Toolbox)', 'Position', [100 100 1150 380]);
42
43 subplot(1,3,1)
44 stem(fk, Ak, 'filled', 'MarkerSize', 3, 'Color', [0.2 0.4 0.8])
45 xlim([0 21]); xlabel('f [Hz]'); ylabel('|w_g| [m/s]')
46 title('INPUT: multisine (flat)'); grid on
47
48 subplot(1,3,2)
49 semilogy(fc, Hh_c, 'Color', [0.85 0.33 0.1], 'LineWidth', 1.8, ...
50 'DisplayName', 'bode (analytical)')
51 hold on
52 scatter(fk, Hh_k, 20, [0.1 0.1 0.5], 'filled', ...
53 'DisplayName', 'bode at f_k')
54 xline(fh_nat, '--', 'Color', [.5 .5 .5], 'HandleVisibility', 'off')
55 xline(ft_nat, '--', 'Color', [.5 .5 .5], 'HandleVisibility', 'off')
56 set(gca, 'XScale', 'log'); xlabel('f [Hz]'); ylabel('|H_h(f)|')
57 title('FILTER: bode magnitude'); legend('Location', 'southwest'); grid on
58
59 subplot(1,3,3)
60 stem(fk, Yamp, 'filled', 'MarkerSize', 3, 'Color', [0.1 0.1 0.1])
61 xline(fh_nat, '--', 'Color', [.5 .5 .5])
62 xline(ft_nat, '--', 'Color', [.5 .5 .5])
63 xlim([0 21]); xlabel('f [Hz]'); ylabel('|h| [mm]')
64 title('OUTPUT: response lines'); grid on
65
66 out_dir = fullfile(here, '..', 'imgs');
67 if ~isfolder(out_dir), mkdir(out_dir); end
68 saveas(gcf, fullfile(out_dir, 'multisine_toolbox_response.png'));

```

```
69 fprintf('Saved.\n');
```

Listing 7: multisine_toolbox_response.m — Toolbox 기반 멀티사인 응답

B.5 4주차: 선형·비선형 스프링 거동 비교 (MATLAB)

스프링에 3차 비선형 항($k_n x^3$)을 추가한 경우와 선형 모델의 응답 차이를 탐구한다. FAA 이산 돌풍 입력과 멀티사인 입력에 대해 ode45로 수치적분하고, 등가 루트 굽힘 모멘트를 지표로 선형·비선형 시스템을 정량 비교한다.

B.5.1 impulse_function_generator.m

FAA 14 CFR §25.341(a) 이산 조율 돌풍(1-cosine 파형)을 생성하는 함수. 돌풍 구배 H 를 스위치하여 최대 응답을 유발하는 임계 H 를 탐색하고 결과를 출력한다.

```
1 function gust = impulse_function_generator(varargin)
2 % impulse_function_generator Certification discrete tuned (1-cosine) gust input.
3 % Generates the FAA 14 CFR 25.341(a) / EASA CS-25.341(a) discrete gust as a
4 % gust-velocity signal w_g(t) [m/s] for the Week-3 2-DOF wing model, sweeps the
5 % gust gradient H to find the critical response, and plots the result.
6 % Rationale, formulae, parameters and references: impulse_function_generator.md
7 %
8 %   gust = impulse_function_generator('standard','scaled') % default
9 %   gust = impulse_function_generator('standard','cs25') % literal CS-25 range
10
11 % only crit gusts visualized by default, so that the plot is not too busy. For more, run impulse_all.m
12
13 cfg = default_config();
14 for i = 1:2:numel(varargin)
15     cfg.(varargin{i}) = varargin{i+1};
16 end
17
18 model = load_model(cfg.model_file);
19 [H, f_gust] = gradient_sweep(cfg, model);
20 Uds = design_gust_velocity(cfg, H, model.U);
21 [t, wg] = build_gust_family(H, Uds, model.U, cfg.fs, cfg.settle);
22 resp = evaluate_response(model, t, wg);
23 crit = find_critical(H, resp);
24 gust = pack_output(t, wg, H, Uds, f_gust, resp, crit, cfg);
25
26 if cfg.plot
27     make_plots(gust, model);
28 end
29 end
30
31 % =====
32
33 function cfg = default_config()
34 cfg.standard = 'scaled';
35 cfg.model_file = '';
36 cfg.fs = 2000;
37 cfg.nH = 25;
38 cfg.settle = 6;
39 cfg.Fg = 1.0;
40 cfg.Uref = [];
41 cfg.band = [0.5 2.0];
42 cfg.plot = true;
43 end
44
45 % -----
46
47 function model = load_model(model_file)
48 if isempty(model_file)
49     here = fileparts(mfilename('fullpath'));
50     cand = {fullfile(here,'wing_2dof.mat')};
51     for c = cand
52         if isfile(c{1}), model_file = c{1}; break; end
53     end
54     if isempty(model_file)
55         error('impulse_function_generator:model', ...
56             'wing_2dof.mat not found next to script or in ../week3.');

```

```

73     otherwise
74         error('impulse_function_generator:standard', ...
75             'cfg.standard must be 'scaled' or 'cs25'.');
76 end
77 f_gust = model.U ./ (2*H);
78 end
79
80 % -----
81
82 function Uds = design_gust_velocity(cfg, H, U)
83 Uref = cfg.Uref;
84 if isempty(Uref)
85     switch lower(cfg.standard)
86     case 'cs25', Uref = 17.07;
87     case 'scaled', Uref = 0.10 * U;
88     end
89 end
90 Uds = Uref * cfg.Fg .* (H/107).^(1/6);
91 end
92
93 % -----
94
95 function [t, wg] = build_gust_family(H, Uds, U, fs, settle)
96 Tg = 2*H/U;
97 tmax = max(Tg) + settle;
98 t = (0:1/fs:tmax)';
99 wg = zeros(numel(t), numel(H));
100 for j = 1:numel(H)
101     in = t <= Tg(j);
102     wg(in,j) = (Uds(j)/2) * (1 - cos(pi*U*t(in)/H(j)));
103 end
104 end
105
106 % -----
107
108 function resp = evaluate_response(model, t, wg)
109 sys = ss(model.A, model.B, model.C_out, model.D_out);
110 nH = size(wg,2);
111 resp.h_peak = zeros(nH,1);
112 resp.th_peak = zeros(nH,1);
113 for j = 1:nH
114     y = lsim(sys, wg(:,j), t);
115     resp.h_peak(j) = max(abs(y(:,1))) * 1000;
116     resp.th_peak(j) = max(abs(y(:,2))) * 180/pi;
117 end
118 end
119
120 % -----
121
122 function crit = find_critical(H, resp)
123 [crit.h_max, ih] = max(resp.h_peak);
124 [crit.th_max, it] = max(resp.th_peak);
125 crit.H_h = H(ih); crit.idx_h = ih;
126 crit.H_th = H(it); crit.idx_th = it;
127 end
128
129 % -----
130
131 function gust = pack_output(t, wg, H, Uds, f_gust, resp, crit, cfg)
132 gust.t = t;
133 gust.wg = wg;
134 gust.H = H(:);
135 gust.Uds = Uds(:);
136 gust.f_gust = f_gust(:);
137 gust.resp = resp;
138 gust.crit = crit;
139 gust.cfg = cfg;
140 end
141
142 % -----
143
144 function make_plots(gust, model)
145 col_h = [0.85 0.33 0.10];
146 col_th = [0.20 0.40 0.80];
147 sys = ss(model.A, model.B, model.C_out, model.D_out);
148
149 figure('Name','Discrete Tuned Gust (25.341)','Position',[100 100 820 780]);
150
151 subplot(3,1,1)
152 show = unique(round(linspace(1, numel(gust.H), 5)));
153 hold on
154 for j = show
155     plot(gust.t, gust.wg(:,j), 'LineWidth',1.1, ...
156         'DisplayName',sprintf('H = %.2f m', gust.H(j)));
157 end
158 plot(gust.t, gust.wg(:,gust.crit.idx_h), 'k', 'LineWidth',2, ...
159     'DisplayName','critical (h)');
160 xlabel('t [s]'); ylabel('w_g [m/s]')
161 title('1-cosine discrete gust family $U(s) = (U_{ds}/2)\cdot[1-\cos(\pi s/H)]$', 'Interpreter','latex');
162 legend('Location','northeast'); grid on; xlim([0 max(gust.t)])
163
164 subplot(3,1,2)

```

```

165 yyaxis left
166 plot(gust.H, gust.resp.h_peak, '-o', 'Color',col_h, 'MarkerSize',3); ylabel('peak |h| [mm]')
167 yyaxis right
168 plot(gust.H, gust.resp.th_peak, '-s', 'Color',col_th, 'MarkerSize',3); ylabel('peak |\theta| [deg]')
169 xlabel('gust gradient H [m]')
170 title(sprintf('Tuning sweep -- critical $H_h$ = %.2f m, $H_{\theta}$ = %.2f m', ...
171             gust.crit.H_h, gust.crit.H_th),'Interpreter','latex');
172 xline(gust.crit.H_h, '--','Color',col_h, 'HandleVisibility','off');
173 xline(gust.crit.H_th, '--','Color',col_th, 'HandleVisibility','off');
174 grid on
175
176 subplot(3,1,3)
177 y = lsim(sys, gust.wg(:,gust.crit.idx_h), gust.t);
178 yyaxis left
179 plot(gust.t, y(:,1)*1000, 'Color',col_h, 'LineWidth',1.5); ylabel('h [mm]')
180 yyaxis right
181 plot(gust.t, rad2deg(y(:,2)), 'Color',col_th, 'LineWidth',1.5); ylabel('\theta [deg]')
182 xlabel('t [s]')
183 title('Response to the critical gust'); grid on
184 end

```

Listing 8: impulse_function_generator.m — FAA 이산 돌풍 생성

B.5.2 multisine_function_generator.m

3주차와 동일한 Schroeder 위상 멀티사인 생성기의 4주차 복사본. nonlinear_response.m이 멀티사인 입력을 구성할 때 호출한다.

```

1 function wg_ts = multisine_function_generator(f0, K, rms_amp, fs, nper)
2 % MULTISINE_FUNCTION_GENERATOR Generate a Schroeder-phase multisine as timeseries.
3 %
4 %   wg_ts = multisine_function_generator()           -- default params
5 %   wg_ts = multisine_function_generator(f0,K,rms,fs,nper)
6 %
7 %   f0      fundamental frequency [Hz]      default 0.25
8 %   K       number of harmonics             default 80
9 %   rms_amp desired RMS amplitude [m/s]     default 1.5
10 %   fs      sample rate [Hz]               default 400
11 %   nper    number of periods              default 8
12
13 if nargin < 1, f0      = 0.25; end
14 if nargin < 2, K       = 80;  end
15 if nargin < 3, rms_amp = 1.5; end
16 if nargin < 4, fs      = 400; end
17 if nargin < 5, nper    = 8;   end
18
19 kk = 1:K;
20 Ak = rms_amp / sqrt(K/2) * ones(1, K); % flat amplitude, RMS = rms_amp
21 phi_k = -pi * kk .* (kk-1) / K; % Schroeder phases (low crest factor)
22
23 dt = 1/fs;
24 N = round(fs * (1/f0) * nper);
25 t = (0:N-1)' * dt;
26 wg = sum(Ak .* sin(2*pi*kk.*t + phi_k), 2);
27
28 wg_ts = timeseries(wg, t, 'Name','w_g');
29 wg_ts.TimeInfo.Units = 's';
30 end

```

Listing 9: multisine_function_generator.m (4주차) — Schroeder 멀티사인 생성

B.5.3 nonlinear_response.m

wing_nl.mat을 불러와 비선형 스프링 운동방정식을 ode45로 수치 적분한다. 입력 인덱스로 임펄스·멀티사인·계단·돌풍 생성기를 선택할 수 있으며, 초기 변위 및 스프링 상수를 파라미터로 지정한다.

```

1 function out = nonlinear_response(input_idx, h0, theta0, k1)
2 % nonlinear_response Time response of the 2-DOF wing with nonlinear springs.
3 %
4 %   out = nonlinear_response(input_idx, h0, theta0, k1)
5 %   input_idx : 1 = impulse | 2 = multisine | 3 = unit step | 4 = gust generator
6 %   h0        : initial plunge [m] (default 0)
7 %   theta0    : initial pitch [rad] (default 0)
8 %   k1        : spring-1 stiffness [N/m] (default: value in wing_nl.mat)
9 %
10 %   Solves M*q'' + C*q' + Ka*q + g(q) = Fgust*u(t)
11 %   with   g(q) the per-spring (linear + cubic) structural restoring force,
12 %          u(t) built from the chosen input by to_forcing (the "input_f_modi").
13 %
14 %   Plots a 4-row figure: input u(t) | h(t) | theta(t) | k1(t) tangent stiffness.
15 %   When knl1 = knl2 = 0 the restoring is linear and k1(t) is flat at k1.
16
17 if nargin < 1, input_idx = 2; end
18 if nargin < 2, h0 = 0; end
19 if nargin < 3, theta0 = 0; end

```

```

20 if nargin < 4, k1 = []; end % [] -> keep wing_nl.mat value
21
22 % ---- fixed run settings -----
23 t_end = 8; % simulation length [s]
24 fs = 2000; % sample rate for u(t) [Hz]
25 mag = 1.0; % magnitude (step / impulse)
26 t0 = 0.05; % onset time [s]
27
28 P = load_plant(k1); % matrices + spring params
29 u = to_forcing(input_idx, t_end, fs, mag, t0); % input_f_modi: u(t) interpolant
30
31 % ---- nonlinear equation of motion -----
32 g = @(q) spring_force(q, P);
33 rhs = @(t,x) [ x(3:4);
34 P.M \ ( P.Fgust*u(t) - P.C*x(3:4) - P.Ka*x(1:2) - g(x(1:2)) ) ];
35
36 x0 = [h0; theta0; 0; 0];
37 opts = odeset('RelTol',1e-7, 'AbsTol',1e-9, 'MaxStep',5/fs);
38 [t, x] = ode45(rhs, [0 t_end], x0, opts);
39
40 out = pack(t, x, u, input_idx, P);
41 plot_response(out);
42 end
43
44 % =====
45
46 function P = load_plant(k1)
47 here = fileparts(mfilename('fullpath'));
48 f = fullfile(here, 'wing_nl.mat');
49 if ~isfile(f)
50 error('nonlinear_response:model', 'wing_nl.mat not found -- run build_ss_nl.m first.');
```

```

51 end
52 S = load(f);
53 P.M = S.M_mat; P.C = S.C_mat; P.Ka = S.Ka; P.Fgust = S.Fgust;
54 P.r1 = S.r1; P.r2 = S.r2;
55 if isempty(k1), k1 = S.k1; end
56 P.k1 = k1; P.k2 = S.k2;
57 P.kn11 = S.kn11; P.kn12 = S.kn12;
58 end
59
60 % -----
61
62 function f = spring_force(q, P)
63 d1 = q(1) + P.r1*q(2); % spring-1 deflection
64 d2 = q(1) + P.r2*q(2); % spring-2 deflection
65 F1 = P.k1*d1 + P.kn11*d1^3;
66 F2 = P.k2*d2 + P.kn12*d2^3;
67 f = [ F1 + F2;
68 P.r1*F1 + P.r2*F2 ]; % generalised force [h; theta]
69 end
70
71 % -----
72
73 function u = to_forcing(choice, t_end, fs, mag, t0)
74 % Convert any input choice into ONE uniform object: u(t) interpolant.
75 tg = (0:1/fs:t_end)';
76
77 switch choice
78 case 1 % impulse: short pulse ~ Dirac, peak = mag
79 w = 5/fs;
80 ug = mag * (tg >= t0 & tg < t0 + w);
81
82 case 2 % multisine (Schroeder phases), as Week 3
83 f0 = 0.25; K = 80; kk = 1:K;
84 amp = 1.5/sqrt(K/2);
85 phi = -pi*kk.*(kk-1)/K;
86 ug = sum(amp .* sin(2*pi*f0*(tg*kk) + phi), 2);
87
88 case 3 % unit step, arbitrary magnitude
89 ug = mag * (tg >= t0);
90
91 case 4 % certification gust (critical column)
92 g = impulse_function_generator('fs',fs, 'plot',false);
93 sel = g.crit.idx_h;
94 Fg = griddedInterpolant(g.t, g.wg(:,sel), 'linear','nearest');
95 ug = Fg(tg);
96
97 otherwise
98 error('nonlinear_response:input', 'input_idx must be 1..4.');
```

```

99 end
100
101 u = griddedInterpolant(tg, ug, 'linear','nearest');
102 end
103
104 % -----
105
106 function out = pack(t, x, u, choice, P)
107 names = {'impulse','multisine','step','gust-generator'};
108 out.t = t;
109 out.h = x(:,1);
110 out.theta = x(:,2);
111 out.u = u(t);

```

```

112 out.input = names{choice};
113
114 delta1 = out.h + P.r1*out.theta; % spring-1 deflection history
115 out.k1eff = P.k1 + 3*P.kn1*delta1.^2; % instantaneous tangent stiffness
116 out.P = P;
117 end
118
119 % -----
120
121 function plot_response(out)
122 col_u = [0.40 0.40 0.40];
123 col_h = [0.85 0.33 0.10];
124 col_th = [0.20 0.40 0.80];
125 col_k = [0.30 0.60 0.30];
126
127 figure('Name','Nonlinear time response','Position',[100 100 780 860]);
128
129 subplot(4,1,1)
130 plot(out.t, out.u, 'Color',col_u, 'LineWidth',1.4)
131 ylabel('u(t) [m/s]'); grid on
132 title(sprintf('Input: %s', out.input))
133
134 subplot(4,1,2)
135 plot(out.t, out.h*1000, 'Color',col_h, 'LineWidth',1.5)
136 yline(0,'Color',[.7 .7 .7]); ylabel('h [mm]'); grid on
137
138 subplot(4,1,3)
139 plot(out.t, rad2deg(out.theta), 'Color',col_th, 'LineWidth',1.5)
140 yline(0,'Color',[.7 .7 .7]); ylabel('\theta [deg]'); grid on
141
142 subplot(4,1,4)
143 plot(out.t, out.k1eff, 'Color',col_k, 'LineWidth',1.5)
144 ylabel('k_1(t) [N/m]'); xlabel('t [s]'); grid on
145 title('Spring-1 tangent stiffness k_1 + 3 k_{n1}\delta_1^2')
146 end

```

Listing 10: nonlinear_response.m — 비선형 스프링 시간 응답 (ode45)

B.5.4 impulse_all.m

선형(lsim)과 비선형(ode45) 모델에 동일한 임펄스·멀티사인 돌풍을 가하여 플런지·피치 응답을 나란히 비교하는 종합 스크립트.

```

1 function gust = impulse_function_generator(varargin)
2 % impulse_function_generator Certification discrete tuned (1-cosine) gust input.
3 % Generates the FAA 14 CFR 25.341(a) / EASA CS-25.341(a) discrete gust as a
4 % gust-velocity signal w_g(t) [m/s] for the Week-3 2-DOF wing model, sweeps the
5 % gust gradient H to find the critical response, and plots the result.
6 % Rationale, formulae, parameters and references: impulse_function_generator.md
7 %
8 % gust = impulse_function_generator('standard','scaled') % default
9 % gust = impulse_function_generator('standard','cs25') % literal CS-25 range
10
11 cfg = default_config();
12 for i = 1:2:numel(varargin)
13     cfg.(varargin{i}) = varargin{i+1};
14 end
15
16 model = load_model(cfg.model_file);
17 [H, f_gust] = gradient_sweep(cfg, model);
18 Uds = design_gust_velocity(cfg, H, model.U);
19 [t, wg] = build_gust_family(H, Uds, model.U, cfg.fs, cfg.settle);
20 resp = evaluate_response(model, t, wg);
21 crit = find_critical(H, resp);
22 gust = pack_output(t, wg, H, Uds, f_gust, resp, crit, cfg);
23
24 if cfg.plot
25     make_plots(gust, model);
26 end
27
28
29 % =====
30
31 function cfg = default_config()
32 cfg.standard = 'scaled';
33 cfg.model_file = '';
34 cfg.fs = 2000;
35 cfg.nH = 25;
36 cfg.settle = 6;
37 cfg.Fg = 1.0;
38 cfg.Uref = [];
39 cfg.band = [0.5 2.0];
40 cfg.plot = true;
41 end
42
43 % -----
44
45 function model = load_model(model_file)
46 if isempty(model_file)

```

```

47     here = fileparts(mfilename('fullpath'));
48     cand = {fullfile(here, 'wing_2dof.mat'), ...
49             fullfile(here, '..', 'week3', 'wing_2dof.mat')};
50     for c = cand
51         if isfile(c{1}), model_file = c{1}; break; end
52     end
53     if isempty(model_file)
54         error('impulse_function_generator:model', ...
55             'wing_2dof.mat not found next to script or in ../week3. ');
56     end
57 end
58 model = load(model_file, 'A', 'B', 'C_out', 'D_out', 'U', 'fn_struct');
59 end
60
61 % -----
62
63 function [H, f_gust] = gradient_sweep(cfg, model)
64 switch lower(cfg.standard)
65     case 'cs25'
66         H = logspace(log10(9.1), log10(107), cfg.nH);
67     case 'scaled'
68         fn = model.fn_struct(:)';
69         f_lo = cfg.band(1) * min(fn);
70         f_hi = cfg.band(2) * max(fn);
71         H = logspace(log10(model.U/(2*f_hi)), log10(model.U/(2*f_lo)), cfg.nH);
72     otherwise
73         error('impulse_function_generator:standard', ...
74             'cfg.standard must be ''scaled'' or ''cs25''. ');
75 end
76 f_gust = model.U ./ (2*H);
77 end
78
79 % -----
80
81 function Uds = design_gust_velocity(cfg, H, U)
82 Uref = cfg.Uref;
83 if isempty(Uref)
84     switch lower(cfg.standard)
85         case 'cs25', Uref = 17.07;
86         case 'scaled', Uref = 0.10 * U;
87     end
88 end
89 Uds = Uref * cfg.Fg .* (H/107).^(1/6);
90 end
91
92 % -----
93
94 function [t, wg] = build_gust_family(H, Uds, U, fs, settle)
95 Tg = 2*H/U;
96 tmax = max(Tg) + settle;
97 t = (0:1/fs:tmax)';
98 wg = zeros(numel(t), numel(H));
99 for j = 1:numel(H)
100     in = t <= Tg(j);
101     wg(in,j) = (Uds(j)/2) * (1 - cos(pi*U*t(in)/H(j)));
102 end
103 end
104
105 % -----
106
107 function resp = evaluate_response(model, t, wg)
108 sys = ss(model.A, model.B, model.C_out, model.D_out);
109 nH = size(wg,2);
110 resp.h_all = zeros(numel(t), nH); % plunge histories, all gusts [mm]
111 resp.th_all = zeros(numel(t), nH); % pitch histories, all gusts [deg]
112 resp.h_peak = zeros(nH,1);
113 resp.th_peak = zeros(nH,1);
114 for j = 1:nH
115     y = lsim(sys, wg(:,j), t);
116     resp.h_all(:,j) = y(:,1) * 1000;
117     resp.th_all(:,j) = y(:,2) * 180/pi;
118     resp.h_peak(j) = max(abs(resp.h_all(:,j)));
119     resp.th_peak(j) = max(abs(resp.th_all(:,j)));
120 end
121 end
122
123 % -----
124
125 function crit = find_critical(H, resp)
126 [crit.h_max, ih] = max(resp.h_peak);
127 [crit.th_max, it] = max(resp.th_peak);
128 crit.H_h = H(ih); crit.idx_h = ih;
129 crit.H_th = H(it); crit.idx_th = it;
130 end
131
132 % -----
133
134 function gust = pack_output(t, wg, H, Uds, f_gust, resp, crit, cfg)
135 gust.t = t;
136 gust.wg = wg;
137 gust.H = H(:);
138 gust.Uds = Uds(:);

```

```

139 gust.f_gust = f_gust(:);
140 gust.resp   = resp;
141 gust.crit   = crit;
142 gust.cfg    = cfg;
143 end
144
145 % -----
146
147 function make_plots(gust, model) %#ok<INUSD>
148 nH = numel(gust.H);
149 show = unique(round(linspace(1, nH, 5))); % representative gradients (same set in all panels)
150 pal = lines(numel(show)); % one colour per shown gust, shared across panels
151
152 figure('Name','Discrete Tuned Gust (25.341)','Position',[100 100 820 800]);
153
154 ax1 = subplot(3,1,1);
155 draw_family(ax1, gust.t, gust.wg, show, pal, gust.H, gust.crit.idx_h, ...
156 'w_g [m/s]', 'i-cosine discrete gust family (critical in bold)', true);
157
158 ax2 = subplot(3,1,2);
159 draw_family(ax2, gust.t, gust.resp.h_all, show, pal, gust.H, gust.crit.idx_h, ...
160 'h [mm]', sprintf('Plunge response (critical H_h = %.2f m)', gust.crit.H_h), false);
161
162 ax3 = subplot(3,1,3);
163 draw_family(ax3, gust.t, gust.resp.th_all, show, pal, gust.H, gust.crit.idx_th, ...
164 '\theta [deg]', sprintf('Pitch response (critical H_\theta = %.2f m)', gust.crit.H_th), false);
165 xlabel(ax3, 't [s]');
166 end
167
168 % -----
169
170 function draw_family(ax, t, Y, show, pal, Hvec, idx_crit, ylab, ttl, do_legend)
171 % Same representative gusts and colours in every panel; the critical one bold black.
172 hold(ax, 'on');
173 for i = 1:numel(show)
174 plot(ax, t, Y(:,show(i)), 'Color',pal(i,:), 'LineWidth',1.0, ...
175 'DisplayName',sprintf('H = %.2f m', Hvec(show(i))));
176 end
177 plot(ax, t, Y(:,idx_crit), 'k', 'LineWidth',2.2, 'DisplayName','critical');
178 yline(ax, 0, 'Color',[.85 .85 .85], 'HandleVisibility','off');
179 ylabel(ax, ylab); grid(ax, 'on'); xlim(ax, [0 max(t)]); title(ax, ttl);
180 if do_legend, legend(ax, 'Location','northeast'); end
181 end

```

Listing 11: impulse_all.m — 선형·비선형 임펄스 응답 비교

B.5.5 moment_response.m

날개 루트 굽힘 모멘트 $M_b = L \cdot \int_0^L h(x) dx$ 의 등가 척도를 임펄스·멀티사인 입력 모두에 대해 선형·비선형 두 경우로 계산하고 출력한다.

```

1 function out = moment_response(L)
2 % moment_response Equivalent root bending moment: impulse & multisine.
3 %
4 % out = moment_response(L)
5 % L : wing length [m] (default 0.50)
6 %
7 % Produces two horizontally-aligned plots.
8 % Each plot shows two curves: constant k (linear) and nonlinear k (cubic).
9
10 if nargin < 1, L = 0.50; end
11
12 P = load_params();
13 t_end = 8;
14 fs = 2000;
15
16 % --- impulse -----
17 [t_i_lin, h_i_lin, th_i_lin] = linear_case(P, 1, t_end, fs);
18 r_imp = nonlinear_response(1); close(gcf);
19
20 M_i_lin = equivalent_moment(h_i_lin, th_i_lin, P, L, false);
21 M_i_nl = equivalent_moment(r_imp.h, r_imp.theta, P, L, true);
22
23 % --- multisine -----
24 [t_m_lin, h_m_lin, th_m_lin] = linear_case(P, 2, t_end, fs);
25 r_ms = nonlinear_response(2); close(gcf);
26
27 M_m_lin = equivalent_moment(h_m_lin, th_m_lin, P, L, false);
28 M_m_nl = equivalent_moment(r_ms.h, r_ms.theta, P, L, true);
29
30 out = struct( ...
31 'L', L, ...
32 't_i_lin', t_i_lin, 'M_i_lin', M_i_lin, ...
33 't_i_nl', r_imp.t, 'M_i_nl', M_i_nl, ...
34 't_m_lin', t_m_lin, 'M_m_lin', M_m_lin, ...
35 't_m_nl', r_ms.t, 'M_m_nl', M_m_nl);
36
37 plot_moments(out);

```

```

38 end
39
40 % =====
41
42 function P = load_params()
43 here = fileparts(mfilename('fullpath'));
44 f = fullfile(here, 'wing_n1.mat');
45 if ~isfile(f)
46     error('moment_response:model', 'wing_n1.mat not found -- run build_ss_n1.m first.');
```

```

47 end
48 P = load(f);
49 P.file = f;
50 end
51
52 % -----
53
54 function [t, h, th] = linear_case(P, input_idx, t_end, fs)
55 % Build the same input signal as nonlinear_response/to_forcing, then lsim.
56 t = (0:1/fs:t_end)';
57 mag = 1.0;
58 t0 = 0.05;
59
60 switch input_idx
61     case 1 % impulse: short rectangular pulse
62         w = 5/fs;
63         ug = mag * double(t >= t0 & t < t0 + w);
64
65     case 2 % multisine (Schroeder phases)
66         f0 = 0.25; K = 80; kk = 1:K;
67         amp = 1.5 / sqrt(K/2);
68         phi = -pi * kk .* (kk-1) / K;
69         ug = sum(amp .* sin(2*pi*f0*(t*kk) + phi), 2);
70 end
71
72 sys = ss(P.A, P.B, P.C_out, P.D_out);
73 y = lsim(sys, ug, t);
74 h = y(:,1);
75 th = y(:,2);
76 end
77
78 % -----
79
80 function M = equivalent_moment(h, th, P, L, n1)
81 d1 = h + P.r1*th;
82 d2 = h + P.r2*th;
83 if n1
84     F1 = P.k1*d1 + P.kn1*d1.^3;
85     F2 = P.k2*d2 + P.kn2*d2.^3;
86 else
87     F1 = P.k1*d1;
88     F2 = P.k2*d2;
89 end
90 Mb = L*(F1 + F2);
91 T = P.r1*F1 + P.r2*F2;
92 M = 0.5*( abs(Mb) + sqrt(Mb.^2 + T.^2) );
93 end
94
95 % -----
96
97 function plot_moments(out)
98 col_lin = [0.20 0.40 0.80]; % blue -- constant k
99 col_n1 = [0.85 0.33 0.10]; % orange -- nonlinear k
100 col_diff = [0.30 0.60 0.30]; % green -- absolute difference
101 labels = {'Constant k', 'Nonlinear k', '|Difference|'};
102
103 % Interpolate nonlinear (ode45 adaptive grid) onto the uniform linear grid
104 diff_i = abs(out.M_i_lin - interp1(out.t_i_n1, out.M_i_n1, out.t_i_lin, 'linear', 'extrap'));
105 diff_m = abs(out.M_m_lin - interp1(out.t_m_n1, out.M_m_n1, out.t_m_lin, 'linear', 'extrap'));
106
107 figure('Name','Equivalent bending moment','Position',[100 100 1100 420]);
108 tiledlayout(1,2,'TileSpacing','compact','Padding','compact');
109
110 nexttile
111 plot(out.t_i_lin, out.M_i_lin, 'Color',col_lin, 'LineWidth',1.5)
112 hold on
113 plot(out.t_i_n1, out.M_i_n1, 'Color',col_n1, 'LineWidth',1.5)
114 plot(out.t_i_lin, diff_i, 'Color',col_diff, 'LineWidth',1.2, 'LineStyle','--')
115 yline(0,'Color',[.7 .7 .7],'HandleVisibility','off'); grid on
116 xlabel('t [s]'); ylabel('M_{eq} [N m]')
117 title(sprintf('Impulse response (L = %.2f m)', out.L))
118 legend(labels, 'Location','northeast')
119
120 nexttile
121 plot(out.t_m_lin, out.M_m_lin, 'Color',col_lin, 'LineWidth',1.5)
122 hold on
123 plot(out.t_m_n1, out.M_m_n1, 'Color',col_n1, 'LineWidth',1.5)
124 plot(out.t_m_lin, diff_m, 'Color',col_diff, 'LineWidth',1.2, 'LineStyle','--')
125 yline(0,'Color',[.7 .7 .7],'HandleVisibility','off'); grid on
126 xlabel('t [s]'); ylabel('M_{eq} [N m]')
127 title('Multisine response')
128 legend(labels, 'Location','northeast')
129

```

```

130 here = fileparts(mfilename('fullpath'));
131 exportgraphics(gcf, fullfile(here, 'nl_moment_response.png'), 'Resolution', 300);
132 fprintf('Saved: nl_moment_response.png\n');
133 end

```

Listing 12: moment_response.m — 등가 루트 굽힘 모멘트

B.5.6 result_plot.m

4주차 해석 전체를 요약하는 5×2 서브플롯 종합 그래프를 생성한다. 임펄스·멀티사인 각 입력에 대해 플런지, 피치, 등가 굽힘 모멘트를 열로 배치하고 선형(파란색)·비선형(주황색) 응답을 겹쳐 표시한다.

```

1 function result_plot()
2 % result_plot 5x2 summary figure for Week-4 nonlinear wing analysis.
3 %
4 % Layout
5 % -----
6 % Col 1 (linear / constant-k) | Col 2 (nonlinear, gust option 4)
7 % Row 1 gust family (impulse_fg sp1) | modified gust input u(t)
8 % Row 2 tuning sweep (impulse_fg sp2) | plunge h(t)
9 % Row 3 critical response (impulse_fg sp3) | pitch theta(t)
10 % Row 4 spring-1 constant k | spring-1 tangent k1(t)
11 % Row 5 equiv. bending moment, const-k | equiv. bending moment, varying-k
12
13 here = fileparts(mfilename('fullpath'));
14
15 % ---- 1. Run source functions (suppress internal figures) -----
16 gust = impulse_function_generator('plot', false);
17
18 nl = nonlinear_response(4);
19 close(gcf);
20
21 mout = moment_response();
22 close(gcf);
23
24 % ---- 2. Linear critical response (for rows 3 and 4) -----
25 mdl = load(fullfile(here, '..', 'week3', 'wing_2dof.mat'), ...
26 'A', 'B', 'C_out', 'D_out');
27 sys_lin = ss(mdl.A, mdl.B, mdl.C_out, mdl.D_out);
28 wg_crit = gust.wg(:, gust.crit.idx_h);
29 y_lin = lsim(sys_lin, wg_crit, gust.t); % columns: h [m], theta [rad]
30
31 % constant spring-1 stiffness (linear assumption: k1 independent of deflection)
32 nlp = load(fullfile(here, 'wing_nl.mat'), 'k1');
33 k1_const = nlp.k1 * ones(size(gust.t));
34
35 % ---- 3. Colours (shared with source scripts) -----
36 col_h = [0.85 0.33 0.10];
37 col_th = [0.20 0.40 0.80];
38 col_k = [0.30 0.60 0.30];
39
40 % ---- 4. Figure -----
41 figure('Name', 'Week-4 Results', 'Position', [80 60 1200 1100]);
42
43 % -- Row 1 (full width): 1-cosine gust family -----
44 subplot(5,2,[1 2])
45 show = unique(round(linspace(1, numel(gust.H), 5)));
46 hold on
47 for j = show
48     plot(gust.t, gust.wg(:,j), 'LineWidth', 1.1, ...
49         'DisplayName', sprintf('H = %.1f m', gust.H(j)));
50 end
51 plot(gust.t, gust.wg(:,gust.crit.idx_h), 'k', 'LineWidth', 2, ...
52     'DisplayName', 'critical H');
53 hold off
54 xlabel('t [s]'); ylabel('w_g [m/s]'); grid on
55 title('1-cosine gust family (U_d_s/2)[1-cos(\pi s/H)]')
56 legend('Location', 'northeast')
57 xlim([0 max(gust.t)])
58
59 % -- Row 2, Col 1: gradient-tuning sweep -----
60 subplot(5,2,3)
61 yyaxis left
62 plot(gust.H, gust.resp.h_peak, '-o', 'Color', col_h, 'MarkerSize', 3, ...
63     'DisplayName', 'peak |h|');
64 ylabel('peak |h| [mm]')
65 yyaxis right
66 plot(gust.H, gust.resp.th_peak, '-s', 'Color', col_th, 'MarkerSize', 3, ...
67     'DisplayName', 'peak |\theta|');
68 ylabel('peak |\theta| [deg]')
69 xlabel('H [m]'); grid on
70 xline(gust.crit.H_h, '--', 'Color', col_h, 'HandleVisibility', 'off')
71 xline(gust.crit.H_th, '--', 'Color', col_th, 'HandleVisibility', 'off')
72 title(sprintf('Tuning sweep: H_h = %.2f m, H_\theta = %.2f m', ...
73     gust.crit.H_h, gust.crit.H_th))
74 legend('Location', 'northeast')
75
76 % -- Row 2, Col 2: nonlinear h(t) -----
77 subplot(5,2,4)

```

```

78 plot(nl.t, nl.h*1000, 'Color',col_h, 'LineWidth',1.5, 'DisplayName','h(t)')
79 yline(0,'Color',[.7 .7 .7],'HandleVisibility','off'); grid on
80 xlabel('t [s]'); ylabel('h [mm]')
81 title('Plunge h(t) (nonlinear gust response)')
82 legend('Location','northeast')
83
84 % -- Row 3, Col 1: linear critical response (h and theta) -----
85 subplot(5,2,5)
86 yyaxis left
87 plot(gust.t, y_lin(:,1)*1000, 'Color',col_h, 'LineWidth',1.5, ...
88      'DisplayName','h');
89 ylabel('h [mm]')
90 yyaxis right
91 plot(gust.t, rad2deg(y_lin(:,2)), 'Color',col_th, 'LineWidth',1.5, ...
92      'DisplayName','\theta');
93 ylabel('\theta [deg]')
94 xlabel('t [s]'); grid on
95 title('Linear response to critical gust')
96 legend('Location','northeast')
97
98 % -- Row 3, Col 2: nonlinear theta(t) -----
99 subplot(5,2,6)
100 plot(nl.t, rad2deg(nl.theta), 'Color',col_th, 'LineWidth',1.5, ...
101      'DisplayName','\theta(t)')
102 yline(0,'Color',[.7 .7 .7],'HandleVisibility','off'); grid on
103 xlabel('t [s]'); ylabel('\theta [deg]')
104 title('Pitch \theta(t) (nonlinear gust response)')
105 legend('Location','northeast')
106
107 % -- Row 4, Col 1: constant spring stiffness (linear assumption) -----
108 subplot(5,2,7)
109 plot(gust.t, k1_const, 'Color',col_k, 'LineWidth',1.5, ...
110      'DisplayName','k_1 (const)')
111 xlabel('t [s]'); ylabel('k_1 [N/m]'); grid on
112 title('Spring-1 stiffness (linear: k_1 = const)')
113 legend('Location','northeast')
114
115 % -- Row 4, Col 2: nonlinear tangent stiffness k1eff(t) -----
116 subplot(5,2,8)
117 plot(nl.t, nl.k1eff, 'Color',col_k, 'LineWidth',1.5, ...
118      'DisplayName','k_1(t)')
119 xlabel('t [s]'); ylabel('k_1(t) [N/m]'); grid on
120 title('Spring-1 tangent stiffness k_1 + 3k_n_l_1\delta_1^2')
121 legend('Location','northeast')
122
123 % -- Row 5, Col 1: equivalent bending moment - constant k -----
124 subplot(5,2,9)
125 plot(mout.t_lin, mout.M_lin, 'Color',col_h, 'LineWidth',1.5, ...
126      'DisplayName','M_e_q')
127 yline(0,'Color',[.7 .7 .7],'HandleVisibility','off'); grid on
128 xlabel('t [s]'); ylabel('M_e_q [N m]')
129 title(sprintf('Equiv. bending moment - constant k (L = %.2f m)', mout.L))
130 legend('Location','northeast')
131
132 % -- Row 5, Col 2: equivalent bending moment - varying k -----
133 subplot(5,2,10)
134 plot(mout.t_nl, mout.M_nl, 'Color',col_h, 'LineWidth',1.5, ...
135      'DisplayName','M_e_q')
136 yline(0,'Color',[.7 .7 .7],'HandleVisibility','off'); grid on
137 xlabel('t [s]'); ylabel('M_e_q [N m]')
138 title('Equiv. bending moment - varying k (cubic springs)')
139 legend('Location','northeast')
140
141 end

```

Listing 13: result_plot.m — 4주차 종합 결과 그래프

B.6 5주차: 속도에 따른 고유값 변화 및 플러터 경계 (MATLAB)

비행속도 U 가 변함에 따라 시스템 행렬 $\mathbf{A}(U)$ 의 고유값이 이동하는 양상을 root-locus로 추적하고, 무게중심-공력중심 간격 d_{cg}^{ac} 에 따른 플러터 경계를 2D/3D로 시각화한다.

B.6.1 root_locus_U.m

속도 U 를 근·영에서 최대값까지 스윕하면서 각 U 에서 $\mathbf{A}(U)$ 의 고유값을 계산하여 복소평면에 궤적을 그린다. 허수축 ($\text{Re} = 0$)을 넘는 임계 속도가 플러터 속도 U_f 로 표시된다.

```

1 function root_locus_U()
2 % Root locus of the 2-DOF aeroelastic wing as aero.U varies.
3 %
4 % Sweeps freestream speed U from near-zero to U_max, builds the linear
5 % A-matrix at each U (same parameters as build_2dof in build_models.m),
6 % tracks the four eigenvalue branches, locates the flutter speed where
7 % a branch crosses the imaginary axis (Re = 0, Im ≠ 0), and saves a plot.
8 %
9 % ---- physical parameters (must match build_2dof / aero_terms) -----

```

```

10 m = 1.5;      Ip = 0.0084;      S = 0.0225;
11 k = 5500.0;  cd = 1.0;      d = 0.22;
12
13 rho = 1.225;  chord = 0.30;
14 span = 0.50;  Clalpha = 6;
15 ea = 0.045;
16 U_nom = 12.0; % nominal speed from aero_terms [m/s]
17
18 M_mat = [m, S; S, Ip];
19 Ks = [2*k, 0; 0, k*d^2/2];
20 C_struct = [2*cd, 0; 0, cd*d^2/2];
21
22 % ---- velocity sweep -----
23 U_vec = linspace(0.5, 50, 4000);
24 N = numel(U_vec);
25
26 ev_mat = zeros(4, N); % each column: eigenvalues sorted by imag part
27
28 for i = 1:N
29     U = U_vec(i);
30     qS = 0.5 * rho * U^2 * chord * span;
31
32     Ka = [0, qS*Clalpha; 0, -qS*Clalpha*ea];
33     Ca = (qS*Clalpha / U) .* [1, 0; -ea, 0];
34
35     Minv = inv(M_mat);
36     A = [zeros(2), eye(2);
37          -Minv*(Ks+Ka), -Minv*(C_struct+Ca)];
38
39     ev = eig(A);
40     [~, idx] = sort(imag(ev)); % ascending imag → consistent branch order
41     ev_mat(:, i) = ev(idx);
42 end
43
44 % ---- find flutter speed: first U where max Re crosses 0 from below ----
45 max_re = max(real(ev_mat), [], 1);
46 cross = find(diff(sign(max_re)) > 0, 1);
47
48 U_fl = NaN;
49 im_fl = NaN;
50 if ~isempty(cross)
51     % linear interpolation between the two straddle points
52     U_fl = interp1(max_re(cross:cross+1), U_vec(cross:cross+1), 0);
53
54     % find which branch is the one crossing
55     for b = 1:4
56         re_b = real(ev_mat(b, :));
57         if re_b(cross) < 0 && re_b(cross+1) >= 0
58             im_fl = interp1(U_vec(cross:cross+1), ...
59                             imag(ev_mat(b, cross:cross+1)), U_fl);
60             break
61         end
62     end
63
64     fprintf('Flutter speed:      U_flutter = %.4f m/s\n', U_fl);
65     fprintf('Flutter frequency: %.4f rad/s (%.4f Hz)\n', ...
66             abs(im_fl), abs(im_fl)/(2*pi));
67 else
68     fprintf('No flutter detected in U = [%.1f, %.1f] m/s.\n', ...
69             U_vec(1), U_vec(end));
70 end
71
72 % ---- plot -----
73 figure('Name', 'Root Locus vs aero.U', 'Position', [80 80 950 680]);
74 hold on; grid on; ax = gca;
75
76 colors = lines(4);
77 branch_labels = {'Mode 1 (lower \omega)', 'Mode 2 (upper \omega)', ...
78                 'Mode 2 (conj.)', 'Mode 1 (conj.)'};
79
80 for b = 1:4
81
82     plot(real(ev_mat(b,:)), imag(ev_mat(b,:)), '-', ...
83          'Color', colors(b,:), 'LineWidth', 1.8, ...
84          'DisplayName', branch_labels{b});
85
86     %{
87     plot(real(ev_mat(b,:)), imag(ev_mat(b,:)), '-', ...
88          'Color', colors(b,:), 'LineWidth', 1.8);
89     %}
90
91     % circle at low-U end
92     plot(real(ev_mat(b,1)), imag(ev_mat(b,1)), 'o', ...
93          'Color', colors(b,:), 'MarkerFaceColor', colors(b,:), ...
94          'MarkerSize', 6, 'HandleVisibility', 'off');
95
96     % arrow mid-branch to show direction of increasing U
97     mid = round(N/2);
98     quiver(real(ev_mat(b, mid)), imag(ev_mat(b, mid)), ...
99            real(ev_mat(b, mid+1) - ev_mat(b, mid)), ...
100            imag(ev_mat(b, mid+1) - ev_mat(b, mid)), ...
101            5, 'Color', colors(b,:), 'MaxHeadSize', 3, ...
102            'HandleVisibility', 'off');
103 end

```

```

102
103 % imaginary axis
104 xline(0, 'k--', 'LineWidth', 1.4, 'HandleVisibility', 'off');
105 text(0.5, ax.YLim(1)*0.92, 'Im axis', 'FontSize', 9, 'Color', [0.3 0.3 0.3]);
106
107 %{
108 % nominal operating point (U = 12 m/s)
109 nom_idx = round(interp1(U_vec, 1:N, U_nom));
110 for b = 1:4
111     plot(real(ev_mat(b, nom_idx)), imag(ev_mat(b, nom_idx)), 'ks', ...
112          'MarkerSize', 8, 'LineWidth', 1.5, 'HandleVisibility', 'off');
113 end
114 % one legend entry for the nominal marker
115 plot(nan, nan, 'ks', 'MarkerSize', 8, 'LineWidth', 1.5, ...
116      'DisplayName', sprintf('Nominal U = %.0f m/s', U_nom));
117 %}
118
119 % flutter marker
120 if ~isnan(U_fl)
121     plot(0, im_fl, 'rp', 'MarkerSize', 18, 'MarkerFaceColor', 'r', ...
122          'LineWidth', 1.2, ...
123          'DisplayName', sprintf('Flutter U_{fl} = %.2f m/s', U_fl));
124     plot(0, -im_fl, 'rp', 'MarkerSize', 18, 'MarkerFaceColor', 'r', ...
125          'HandleVisibility', 'off');
126     text(0.4, im_fl, sprintf(' U_{fl} = %.2f m/s', U_fl), ...
127          'FontSize', 11, 'Color', 'r', 'FontWeight', 'bold');
128     text(0.4, -im_fl, sprintf(' U_{fl} = %.2f m/s', U_fl), ...
129          'FontSize', 11, 'Color', 'r', 'FontWeight', 'bold');
130 end
131
132 xlabel('Real part \sigma [1/s]', 'FontSize', 12);
133 ylabel('Imaginary part \omega_d [rad/s]', 'FontSize', 12);
134 title(['Root Locus w.r.t. aero.U -- 2-DOF aeroelastic wing'], ...
135       'FontSize', 12);
136 legend('Location', 'best', 'FontSize', 9);
137
138 here = fileparts(mfilename('fullpath'));
139 out = fullfile(here, 'root_locus_U.png');
140 %exportgraphics(gcf, out, 'Resolution', 300);
141 fprintf('Figure saved: %s\n', out);
142 end

```

Listing 14: root_locus_U.m — 속도에 따른 root-locus

B.6.2 flutter_vs_d.m

무계중심-공력중심 간격 d_{cg}^{ac} 값을 변화시키면서(e_a 는 고정) 각각의 플러터 속도 U_f 를 계산하여 2D 선 그래프와 3D 안정성 곡면(max Re(λ) vs (d_{cg}^{ac}, U))을 생성한다. 플러터 경계를 최고 비행속도 이상으로 확보하기 위한 최적 d_{cg}^{ac} 설계 근거를 제공한다.

```

1 %function [d_cm,U_fl] = flutter_vs_d()
2 function flutter_vs_d()
3 % Flutter speed and stability surface as a function of CG-to-AC distance d.
4 %
5 % d = x_CG - x_AC [m]: + when CG is aft of AC (destabilising)
6 %                      - when CG is forward of AC (stabilising)
7 %
8 % NACA 2412 context: AC at ~25% chord; user specifies CG also at 25% chord
9 % => d_nom = 0, which maps to S = m*(0 - ea) = -m*ea = -0.10 kg.m.
10 %
11 % Relation to build_2dof parameters (build_models.m):
12 % S = m * (d - ea) static unbalance [kg.m]
13 % M_mat = [m, S; S, Ip] mass matrix
14 %
15 % Physical constraint for PD mass matrix:
16 % m*Ip > S^2 => |d - ea| < sqrt(Ip/m) = r_gyr
17 %
18 % Outputs:
19 % Figure 1 2-D line: U_flutter vs d
20 % Figure 2 3-D surf: max Re(lambda) over (d, U) space
21 % Red curve on surface = flutter boundary (Re = 0)
22
23 % ---- fixed parameters (match build_2dof / aero_terms in build_models.m) --
24 m = 1.5; Ip = 0.0084;
25 k = 5500.0; cd = 1.0; d_sp = 0.22; % d_sp = spring spacing (not CG-AC d!)
26
27 rho = 1.225; chord = 0.30; span = 0.50;
28 Clalpha = 6; ea = 0.045; % AC forward of elastic axis [m]
29
30 % Structural matrices (independent of d or U)
31 Ks = [2*k, 0; 0, k*d_sp^2/2];
32 C_struct = [2*cd, 0; 0, cd*d_sp^2/2];
33
34 % ---- parametric range -----
35 % PD constraint: |d - ea| < sqrt(Ip/m)
36 r_gyr = sqrt(Ip / m); % = 0.0707 m
37 d_min = ea - r_gyr + 5e-4; % m (add small margin)
38 d_max = ea + r_gyr - 5e-4;

```

```

39
40 d_vec = linspace(d_min, d_max, 200); % CG-to-AC distance [m]
41 U_vec = linspace(0.5, 50, 500); % freestream speed [m/s]
42
43 Nd = numel(d_vec);
44 Nu = numel(U_vec);
45
46 max_re_surf = nan(Nu, Nd); % max Re(eig(A)) at each (U, d) point
47 U_fl = nan(1, Nd); % flutter speed for each d
48
49 for id = 1:Nd
50     d_cga = d_vec(id);
51     S = m * (d_cga - ea); % static unbalance
52     M = [m, S; S, Ip];
53
54     if det(M) <= 0; continue; end
55     Minv = inv(M);
56
57     for iu = 1:Nu
58         U = U_vec(iu);
59         qS = 0.5 * rho * U^2 * chord * span;
60
61         Ka = [0, qS*Clalpha; 0, -qS*Clalpha*ea];
62         Ca = (qS*Clalpha / U) .* [1, 0; -ea, 0];
63
64         A = [zeros(2), eye(2);
65             -Minv*(Ks + Ka), -Minv*(C_struct + Ca)];
66
67         max_re_surf(iu, id) = max(real(eig(A)));
68     end
69
70     % flutter crossing: first U where max Re crosses 0 from below
71     col = max_re_surf(:, id);
72     cx = find(diff(sign(col)) > 0, 1);
73     if ~isempty(cx)
74         U_fl(id) = interp1(col(cx:cx+1), U_vec(cx:cx+1), 0);
75     end
76 end
77
78 d_cm = d_vec * 100; % display in cm
79
80 % print summary
81 fprintf('CG-to-AC distance range: [%.2f, %.2f] cm\n', d_cm(1), d_cm(end));
82 fprintf('d = 0 (CG at AC, NACA2412 nom.): U_flutter = %.3f m/s\n', ...
83     interp1(d_cm, U_fl, 0, 'linear', NaN));
84 [min_Ufl, i_min] = min(U_fl);
85 fprintf('Lowest flutter: U_fl = %.3f m/s at d = %.2f cm\n', min_Ufl, d_cm(i_min));
86
87 % =====
88 % Figure 1 -- 2-D: U_flutter vs d
89 % =====
90 f1 = figure('Name','Flutter speed vs d', 'Position',[60 80 760 470]);
91 plot(d_cm, U_fl, 'b-', 'LineWidth', 2.5);
92 hold on; grid on;
93
94 % mark d = 0 (CG at AC)
95 xline(0, 'k--', 'LineWidth', 1.3, 'HandleVisibility','off');
96 text(0.3, min(U_fl, [], 'omitnan') * 0.93, ...
97     ' d = 0 (CG at AC)', 'FontSize', 9, 'Color', [0.3 0.3 0.3]);
98
99 % mark original model point (d from original S = 0.015)
100 S_orig = 0.015;
101 d_orig = S_orig/m + ea; % = 0.0575 m = 5.75 cm
102 Ufl_orig = interp1(d_cm, U_fl, d_orig*100, 'linear', NaN);
103 if ~isnan(Ufl_orig)
104     plot(d_orig*100, Ufl_orig, 'rs', 'MarkerSize', 10, 'MarkerFaceColor','r', ...
105         'DisplayName', sprintf('build\\_2dof d=%.1fcm U_{fl}=%.1f m/s', ...
106             d_orig*100, Ufl_orig));
107     legend('Location','best','FontSize',9);
108 end
109
110 xlabel('d = x_{CG} - x_{AC} [cm] (+ : CG aft of AC)', 'FontSize', 12);
111 ylabel('Flutter speed U_{flutter} [m/s]', 'FontSize', 12);
112 title({'Flutter speed vs CG-to-AC distance d', ...
113     'NACA 2412 -- CG at 25% chord when d = 0'}, 'FontSize', 12);
114
115 % =====
116 % Figure 2 -- 3-D surf: max Re(lambda) over (d, U)
117 % =====
118 [D_cm, UU] = meshgrid(d_cm, U_vec);
119
120 f2 = figure('Name','3D stability surface (d, U)', 'Position',[130 80 1000 680]);
121
122 % surface
123 surf(D_cm, UU, max_re_surf, 'EdgeColor','none', 'FaceAlpha', 0.88);
124 hold on;
125
126 % flutter boundary: Re = 0 contour on the surface (red)
127 contour3(D_cm, UU, max_re_surf, [0 0], 'r-', 'LineWidth', 3.5);
128
129 % transparent Re = 0 plane for reference
130 xlm = [min(d_cm), max(d_cm)];

```

```

131 ylm = [U_vec(1), U_vec(end)];
132 patch([xlm(1) xlm(2) xlm(2) xlm(1)], ...
133       [ylm(1) ylm(1) ylm(2) ylm(2)], ...
134       [0 0 0], [0.6 0.6 0.6], ...
135       'FaceAlpha', 0.12, 'EdgeColor', 'none');
136
137 % original model marker projected onto surface
138 if ~isnan(Ufl_orig) && d_orig*100 >= d_cm(1) && d_orig*100 <= d_cm(end)
139     z_marker = interp2(D_cm, UU, max_re_surf, d_orig*100, Ufl_orig, 'linear', NaN);
140     plot3(d_orig*100, Ufl_orig, 0, 'rs', 'MarkerSize', 12, ...
141           'MarkerFaceColor', 'r', 'LineWidth', 1.5);
142 end
143
144 % colormap: blue (stable) -> white (neutral) -> red (unstable)
145 colormap(bwr_cmap(256));
146 c_lim = prctile(abs(max_re_surf(:)), 95);
147 clim([-c_lim, c_lim]);
148 cb = colorbar;
149 cb.Label.String = 'max Re(\lambda) [1/s]';
150 cb.Label.FontSize = 11;
151
152 xlabel('d = x_{CG} - x_{AC} [cm]', 'FontSize', 11);
153 ylabel('Freestream speed U [m/s]', 'FontSize', 11);
154 zlabel('max Re(\lambda) [1/s]', 'FontSize', 11);
155 title(['Stability surface over CG-to-AC distance d, speed U'], ...
156       'Red curve = flutter boundary [Re(\lambda) = 0]', 'FontSize', 12);
157 view([-42, 28]);
158
159 % ---- save -----
160 %here = fileparts(mfilename('fullpath'));
161 %exportgraphics(f1, fullfile(here, 'flutter_vs_d_2D.png'), 'Resolution', 300);
162 %exportgraphics(f2, fullfile(here, 'flutter_vs_d_3D.png'), 'Resolution', 300);
163 %fprintf('Saved: flutter_vs_d_2D.png and flutter_vs_d_3D.png\n');
164 end
165
166 % =====
167 % Local helper: diverging blue-white-red colormap
168 % =====
169 function cmap = bwr_cmap(n)
170 n2 = ceil(n / 2);
171 blue_to_white = [linspace(0,1,n2)', linspace(0,1,n2)', ones(n2,1)];
172 white_to_red = [ones(n2,1), linspace(1,0,n2)', linspace(1,0,n2)'];
173 cmap = [blue_to_white; white_to_red];
174 cmap = cmap(1:n, :);
175 end

```

Listing 15: flutter_vs_d.m — 간격 d_{cg}^{ac} 에 따른 플러터 경계

References

- [1] J.D. Anderson and C.P. Cadou. *Fundamentals of Aerodynamics*. McGraw-Hill series in aeronautical and aerospace engineering. McGraw Hill, 2023. ISBN: 9781266076442. URL: <https://books.google.co.kr/books?id=PUV7zwEACAAJ>.
- [2] D.H. Hodges and G.A. Pierce. *Introduction to Structural Dynamics and Aeroelasticity*. Cambridge Aerospace Series. Cambridge University Press, 2011. ISBN: 9781139499927. URL: https://books.google.co.kr/books?id=_UCVJ6RY-vsC.
- [3] AS Mendes, PS Meirelles, and DE Zampieri. “Analysis of torsional vibration in internal combustion engines: modelling and experimental validation”. In: *Proceedings of the Institution of Mechanical Engineers, Part K: Journal of Multi-body Dynamics* 222.2 (2008), pp. 155–178.
- [4] Brian L. Stevens, Frank L. Lewis, and Eric N. Johnson. “The Kinematics and Dynamics of Aircraft Motion”. In: *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*. John Wiley & Sons, Ltd, 2015. Chap. 1, pp. 1–62. ISBN: 9781119174882. DOI: <https://doi.org/10.1002/9781119174882.ch1>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119174882.ch1>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119174882.ch1>.